



Università degli Studi di Salerno

Dipartimento di Informatica

Corso di Laurea Triennale

Progettazione e Sviluppo di un Sistema Edge AI per il Tracciamento Video Automatico in Ambito Sportivo

Relatore

Prof. Fabio Narducci

Candidato

Lorenzo Olivola

Matricola: 0512119024

Anno Accademico 2025/2026

Indice

Abstract	1
1 Introduzione	2
1.1 Problema Affrontato	2
1.2 Soluzioni Esistenti e Limiti	3
1.3 Configurazione Scelta	3
2 Stato dell'Arte	5
2.1 Internet of Things (IoT)	5
2.2 Object Detection	5
2.3 Intelligenza Artificiale e Machine Learning	6
2.4 Classificazione e Reti Neurali	7
2.5 Principali Architetture per l'Object Detection	7
2.6 Dispositivi Commerciali Esistenti	9
2.7 Dispositivi Non Commerciali Esistenti	14
3 Metodologia	19
3.1 Architettura e flusso dei dati	19
3.2 Metodologia Hardware	20
3.2.1 Raspberry Pi 5	20
3.2.2 AI HAT+	22
3.2.3 Raspberry Pi 5 Active Cooler	23
3.2.4 Micro servomotore SG90 9g	25
3.2.5 Modulo Camera FTK0056B IMX219	26
3.3 Metodologia Software	28
3.3.1 Modello YOLOv8n	28
3.3.2 Modulo per l'object detection	30
3.3.3 Modulo Gestione Telecamera	33
3.3.4 Modulo Gestione Rotori	35
3.3.5 Modulo Gestione Telecamera + Rotori	36
3.3.6 Algoritmo Finale	38

4	Casi e Risultati	42
4.1	Metriche prestazionali: Analisi termica e FPS	42
4.2	Casi Sperimentati	45
5	Conclusioni	50
5.1	Limiti della Ricerca	50
5.2	Sviluppi Futuri	50
5.3	Conclusioni	51

Abstract

Il presente lavoro di tesi illustra la progettazione e lo sviluppo di un sistema IoT di edge computing per il tracciamento video automatizzato, finalizzato all'analisi tattica e sportiva con particolare riferimento alla pallacanestro. L'obiettivo primario è superare le limitazioni delle attuali soluzioni commerciali, le quali risultano frequentemente caratterizzate da costi proibitivi, analisi differite o forte dipendenza da infrastrutture di rete complesse.

Il sistema si basa su un Raspberry Pi 5, integrato con un modulo AI HAT+ dotato di NPU Hailo-8 per l'accelerazione neurale in locale. Il flusso video è acquisito tramite un modulo camera IMX219. L'orientamento autonomo dell'inquadratura è garantito da due micro servomotori SG90, impiegati per la gestione degli assi orizzontale e verticale. L'elaborazione in tempo reale è affidata al modello predittivo YOLOv8n, ottimizzato per dispositivi embedded. L'algoritmo filtra le rilevazioni isolando unicamente la classe "persona". Il software calcola il centroide medio dei soggetti inquadrati. Le coordinate estratte alimentano un controllore logico (basato su parametri Proporzionali, Integrali e Derivativi) che converte l'errore di posizione in segnali PWM per azionare dinamicamente i servomotori.

L'analisi prestazionale dimostra l'elevata efficienza della soluzione proposta: l'architettura eterogenea mantiene un framerate costante di 30 FPS e un regime termico controllato (picco massimo di $60.0^{\circ}C$), scongiurando criticità legate al thermal throttling. Nonostante alcune limitazioni emerse durante i test sui movimenti ad alta velocità, causate dai vincoli meccanici dei servomotori economici e dalla classificazione generica dei soggetti, il prototipo conferma la fattibilità di un ecosistema di analisi video autonomo, a bassa latenza e scalabile. Le implementazioni future prevedono l'integrazione di motori passo-passo ad alte prestazioni e il fine-tuning del modello neurale per il riconoscimento specifico della palla e delle diverse squadre in campo.

Capitolo 1

Introduzione

Negli ultimi anni, l'analisi delle prestazioni sportive ha subito una profonda trasformazione, diventando uno strumento fondamentale per atleti e staff tecnici al fine di ottimizzare gli allenamenti e le strategie di gara. Nelle varie discipline sportive, tuttavia, la necessità di monitorare le dinamiche in campo si scontra spesso con i limiti delle soluzioni attuali, le quali risultano frequentemente costose, limitate al tracciamento puramente fisico, vincolate a regie televisive o incapaci di fornire riscontri tattici in tempo reale. Per rispondere a queste problematiche, questa tesi illustra la progettazione e lo sviluppo di un sistema IoT (Internet of Things) di edge computing, basato sull'intelligenza artificiale e sull'object detection, capace di orientare autonomamente una telecamera per seguire in tempo reale le azioni di una competizione sportiva. Il presente elaborato introduce una panoramica sullo stato dell'arte delle tecnologie di visione artificiale (come le Reti Neurali Convoluzionali e le architetture YOLO) e delle soluzioni per l'analisi sportiva già esistenti. Successivamente, il lettore verrà guidato attraverso la metodologia hardware e software del progetto, scoprendo come l'integrazione di componenti accessibili — come un Raspberry Pi 5, un acceleratore neurale AI HAT+, motori a rotazione e il modello predittivo YOLOv8n — abbia permesso di realizzare un ecosistema automatizzato ed efficiente, culminando infine con la presentazione dei risultati ottenuti dal sistema.

1.1 Problema Affrontato

L'obiettivo è realizzare un sistema IoT (Internet of Things) basato sull'uso dell'intelligenza artificiale e di dispositivi visivi, capace di seguire in tempo reale le azioni di una competizione sportiva. Ciò permetterebbe agli atleti e allo staff tecnico di analizzare i movimenti in campo, sia della propria squadra sia degli avversari, al fine di migliorare le prestazioni durante gli allenamenti. Inoltre, l'utilizzo di un sistema di questo tipo durante le gare fornirebbe agli allenatori una visione più ampia dell'incontro, consentendo loro di intervenire tempestivamente con tattiche e consigli mirati per ottimizzare lo svolgimento del match.

L'impiego congiunto dell'Internet of Things e dell'intelligenza artificiale per l'analisi in ambito sportivo è motivato da una serie di vantaggi fondamentali:

- **Automazione del tracciamento e gestione autonoma:** l'integrazione dell'IA all'interno di dispositivi IoT elimina la necessità di un operatore umano per la gestione o l'orientamento della telecamera, demandando l'acquisizione dei dati a nodi capaci di interagire autonomamente con l'ambiente fisico.
- **Analisi in tempo reale (Edge Computing)[1]:** la capacità di elaborare flussi video istantaneamente sui dispositivi stessi consente di mantenere l'inquadratura sempre centrata sull'azione principale con una latenza compatibile con il funzionamento in tempo reale.
- **Riconoscimento multiplo simultaneo:** sebbene la tecnologia dell'object detection offra la potenzialità teorica di identificare e seguire contemporaneamente elementi eterogenei sul campo (come giocatori di squadre diverse, palla e arbitri), l'attuale implementazione del prototipo si focalizza esclusivamente sul rilevamento e tracciamento della classe generica "person".
- **Adattabilità e generalizzazione:** grazie al machine learning, il modello pre-addestrato possiede la potenziale capacità di generalizzare a diverse condizioni ambientali (variazioni di illuminazione, tipologie di campo, colori delle divise) senza richiedere una programmazione esplicita o rigida. Tuttavia, non adattandosi automaticamente ai nuovi scenari, tale effettiva capacità di generalizzazione a condizioni diverse deve essere sempre verificata sperimentalmente.

1.2 Soluzioni Esistenti e Limiti

In commercio esistono già soluzioni che permettono l'analisi delle competizioni sportive, le quali possono essere suddivise in due macro-categorie: tracciamento fisico e analisi video. Il tracciamento fisico si avvale di dispositivi indossabili per monitorare parametri atletici e biomeccanici; tuttavia, tali sistemi possono risultare invasivi per l'atleta, faticano a fornire dati spaziali accurati in ambienti indoor e mancano di informazioni tattiche legate alle dinamiche di gioco. L'analisi video, invece, sfrutta riprese ottiche e algoritmi di visione artificiale per estrarre dati posizionali e strategici senza richiedere sensori fisici; di contro, queste piattaforme sono spesso caratterizzate da costi proibitivi per le piccole società, da un'elevata complessità operativa e da una forte dipendenza da infrastrutture di rete stabili o da riprese televisive di tipo broadcast.

1.3 Configurazione Scelta

La soluzione proposta nella presente tesi si colloca nell'ambito dell'analisi video. L'obiettivo è minimizzare i costi, garantire la facilità di utilizzo e favorire il miglioramento degli atleti consentendo l'acquisizione automatizzata di riprese video in tempo reale, successivamente utilizzabili per l'analisi delle performance durante le partite. Per soddisfare tali requisiti, si è optato per l'utilizzo della seguente configurazione hardware:

- **Raspberry Pi 5:** È stato scelto come unità di elaborazione centrale perché offre prestazioni elevate, con un processore 2-3 volte più veloce rispetto al modello precedente e un'interfaccia PCIe 2.0, mantenendo il prezzo contenuto tipico dei dispositivi single-board. Permette di avere un sistema di edge computing potente senza i costi di elaboratori tradizionali.
- **AI HAT+:** È stato implementato per fornire al sistema un chip acceleratore AI (Hailo-8) dedicato. Garantisce l'elevata potenza di calcolo necessaria per eseguire i modelli di intelligenza artificiale localmente e a bassa latenza, evitando di dover ricorrere a costosi server cloud remoti o a costose GPU esterne.
- **Micro servomotore SG90 9g:** Ne sono stati impiegati due per gestire la rotazione orizzontale e l'inclinazione verticale della telecamera. Sono componenti hardware estremamente economici, ma in grado di fornire il controllo angolare e la reattività necessari per muovere autonomamente l'inquadratura seguendo le azioni di gioco.
- **Modulo Camera IMX219:** Utilizzato per l'acquisizione del flusso video, offre risoluzioni e framerate adeguati a tracciare movimenti rapidi (come 1080p a 30fps o 720p a 60fps) mantenendo un costo molto più accessibile rispetto a telecamere professionali o industriali.
- **Modello YOLOv8n:** È stato selezionato per l'object detection nella sua versione "nano" per ottimizzare al massimo l'hardware a disposizione. Avendo un'impronta di memoria ridotta e un costo computazionale minimo, permette di ottenere prestazioni fluide e in tempo reale sull'hardware embedded economico, scongiurando la necessità di investire in processori più costosi per l'elaborazione software.

Nei capitoli successivi verranno analizzate nel dettaglio le caratteristiche di questa configurazione.

Capitolo 2

Stato dell'Arte

2.1 Internet of Things (IoT)

Nel documento redatto per la **Internet Society** (ISOC) [2] viene evidenziato come, nel panorama scientifico e industriale, non esista ancora una singola definizione universale e rigorosa di **Internet of Things** (IoT), a causa delle innumerevoli declinazioni tecnologiche e applicative del paradigma. Viene tuttavia proposta una definizione architetturale e operativa, che descrive l'IoT come un insieme di scenari in cui la connettività di rete e le capacità di elaborazione si estendono a oggetti, sensori e articoli di uso quotidiano che non sono tradizionalmente considerati elaboratori o computer.

Secondo tale prospettiva, il paradigma si concretizza nella creazione di un ecosistema distribuito in cui questi dispositivi fisici acquisiscono la capacità di generare, scambiare e consumare dati in modo autonomo e con un intervento umano minimo o nullo. Dal punto di vista informatico, questa transizione rappresenta un'evoluzione fondamentale: la rete cessa di essere un'infrastruttura concepita primariamente per connettere dispositivi *end-user* guidati dall'uomo, per trasformarsi in una piattaforma globale in cui oggetti eterogenei interagiscono con l'ambiente fisico circostante. Attraverso sensori e attuatori, i nodi raccolgono informazioni di contesto, le elaborano – localmente o delegandole a infrastrutture cloud – e le utilizzano per orchestrare azioni mirate, abilitando nuovi servizi basati sui dati e l'automazione avanzata dei processi.

2.2 Object Detection

Nel presente progetto vengono adottate tecniche di object detection [3], disciplina che combina la classificazione e la localizzazione spaziale di oggetti di interesse all'interno di contesti visivi statici o dinamici. A differenza del semplice riconoscimento d'immagine, l'object detection identifica e delimita la presenza di ciascuna istanza tramite coordinate geometriche espresse sotto forma di bounding box [4]. Tali metodologie trovano ampio impiego in diversi ambiti applicativi, dalla gestione intelligente della mobilità alla videosorveglianza, fino alla diagnostica per immagini e all'analisi delle prestazioni sportive. Il presente lavoro si inserisce

proprio in questo contesto, ponendosi l'obiettivo di rilevare in modo automatico i giocatori presenti nel campo da gioco durante una partita di basket.

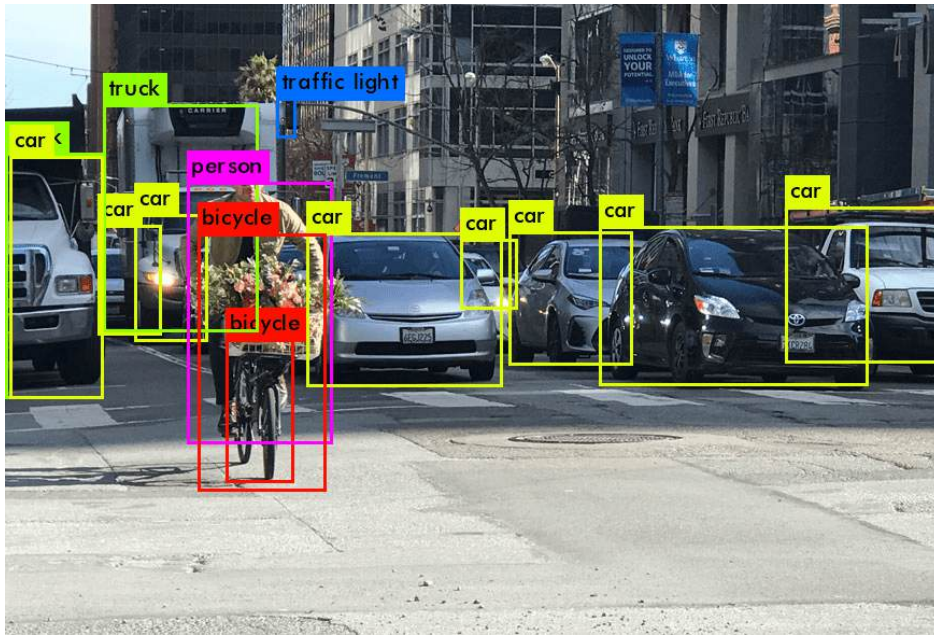


Figura 2.1: Esempio di object detection.

2.3 Intelligenza Artificiale e Machine Learning

L'intelligenza artificiale [5] rappresenta la branca dell'informatica finalizzata alla progettazione e allo sviluppo di sistemi tecnologici capaci di emulare facoltà cognitive tipicamente umane, quali la visione artificiale, l'elaborazione del linguaggio naturale, il ragionamento logico e il supporto ai processi decisionali complessi. Come evidenziato in letteratura, l'obiettivo dell'IA non si limita alla mera esecuzione di algoritmi deterministici, ma si estende alla creazione di agenti capaci di interagire in modo adattivo e autonomo con contesti reali, sollevando al contempo fondamentali questioni etiche e metodologiche legate alla loro responsabilità e trasparenza.

All'interno di questo ampio paradigma, il machine learning [6] costituisce il sottoinsieme metodologico ed ingegneristico fondamentale che abilita l'apprendimento automatico a partire dall'analisi dei dati, superando la necessità di una programmazione esplicita per ogni singola eventualità. Diversamente dai sistemi software tradizionali — strutturati su una logica computazionale rigida, basata su regole e istruzioni scritte manualmente dagli sviluppatori — il machine learning impiega algoritmi di natura statistica ed euristica capaci di identificare autonomamente pattern, correlazioni e regolarità sottostanti all'interno di dataset complessi. In un'ottica sistemica, tale approccio permette ai modelli di generalizzare l'informazione appresa a dati mai visti in precedenza, affinando progressivamente l'accuratezza e le prestazioni del sistema attraverso cicli continui di iterazione, validazione e monitoraggio nell'intero ciclo di vita del software.

2.4 Classificazione e Reti Neurali

Nell'ambito del riconoscimento visivo, bisogna distinguere tra tecniche *supervisionate* e *non supervisionate*. Le prime si basano su dati già etichettati, cioè esempi già noti, mentre le seconde lavorano su dati non classificati cercando in maniera automatica caratteristiche che legano le immagini tra di loro.

Per la realizzazione di modelli in grado di riconoscere oggetti o classificare immagini, una delle tecnologie migliori si basa sulle Reti Neurali Convoluzionali (CNN) [7]. Queste reti sono ispirate al funzionamento del cervello umano e sono indicate principalmente per l'elaborazione delle immagini grazie alla loro capacità di cogliere dettagli visivi su più livelli.

Una CNN è composta dai seguenti strati:

- **Convolutional layer:** è il primo strato di una rete convoluzionale. Utilizza filtri che scorrono sull'immagine per rilevare caratteristiche locali come bordi, angoli e texture. Ogni filtro crea una mappa delle caratteristiche che evidenzia modelli specifici. Il primo strato convoluzionale può essere seguito da un ulteriore strato, rendendo la struttura della CNN gerarchica, in quanto gli strati successivi possono vedere i pixel all'interno dei campi ricettivi degli strati precedenti.
- **Pooling layer:** riduce le dimensioni delle mappe delle caratteristiche riassumendo le aree locali e scartando le informazioni irrilevanti. In questo modo si riduce la complessità computazionale e si conservano le informazioni più importanti.
- **Fully connected layer:** questo strato collega tutti i neuroni tra loro, in modo simile a una rete neurale tradizionale. Viene utilizzato per la classificazione finale e combina le caratteristiche estratte in una decisione fino a identificare l'oggetto desiderato.

Ecco un diagramma di funzionamento di una CNN:

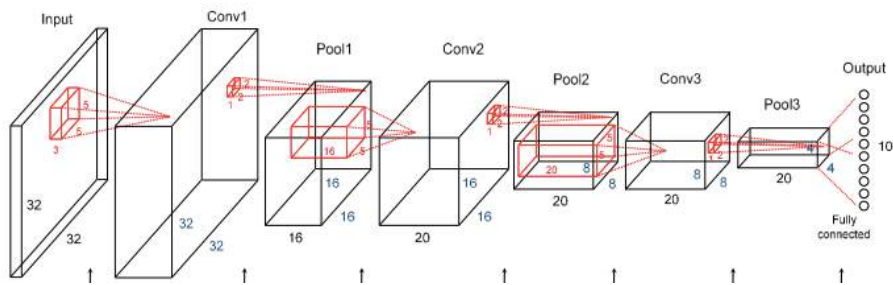


Figura 2.2: Architettura generale di una CNN.

2.5 Principali Architetture per l'Object Detection

Di seguito verranno descritti i due principali tipi di architetture CNN maggiormente discussi nella letteratura scientifica per il rilevamento di oggetti:

- **R-CNN** [8]: L'algoritmo non analizza l'intera immagine tramite finestre scorrevoli, bensì identifica circa 2000 regioni candidate (*bounding box*) che presentano un'alta probabilità di contenere un oggetto, riducendo drasticamente lo spazio di ricerca. Poiché le regioni candidate hanno dimensioni e proporzioni variabili, vengono sottoposte a un processo di ridimensionamento (*warping* o *cropping*) per ottenere una dimensione fissa. Successivamente, ogni regione viene elaborata in modo indipendente da una rete neurale convoluzionale (CNN) pre-addestrata per estrarne le feature rappresentative. Il vettore di *feature* estratto per ciascuna regione viene passato a una serie di *Support Vector Machine* (SVM) lineari, addestrate in modo specifico per classificare la presenza o l'assenza delle varie categorie di oggetti. In parallelo, un modello di regressione lineare perfeziona le coordinate del bounding box per farlo aderire con maggiore precisione all'oggetto rilevato.

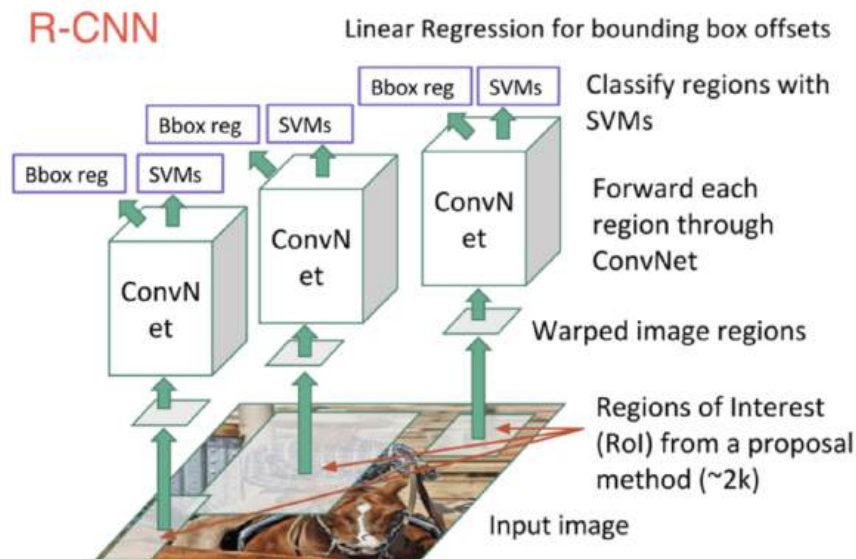


Figura 2.3: Una panoramica di come funziona R-CNN.

- **YOLO** [9]: L'immagine di input viene divisa in una griglia di dimensioni $S \times S$. Se il centro geometrico di un oggetto cade all'interno di una specifica cella, tale cella diventa l'unica responsabile del rilevamento di quell'oggetto. Ogni singola cella predice un numero fisso B di bounding box. Per ogni riquadro, il modello restituisce cinque parametri fondamentali: le coordinate del centro spaziale (x, y), la larghezza (w), l'altezza (h) e un punteggio di confidenza (*confidence score*). Questo punteggio rappresenta la probabilità che il riquadro contenga effettivamente un oggetto, moltiplicata per l'accuratezza del riquadro stesso. Parallelamente alla regressione spaziale, ogni cella della griglia predice un insieme di probabilità C condizionate, le quali indicano a quale classe appartenga l'oggetto individuato (es. "persona", "veicolo", "animale"). Poiché l'approccio a griglia genera inevitabilmente molteplici riquadri sovrapposti per un medesimo oggetto, YOLO applica un algoritmo chiamato *Non-Maximum Suppression* (NMS). Questo meccanismo

confronta i vari bounding box sovrapposti ed elimina le previsioni ridondanti aventi confidenza minore, mantenendo esclusivamente il riquadro più preciso.

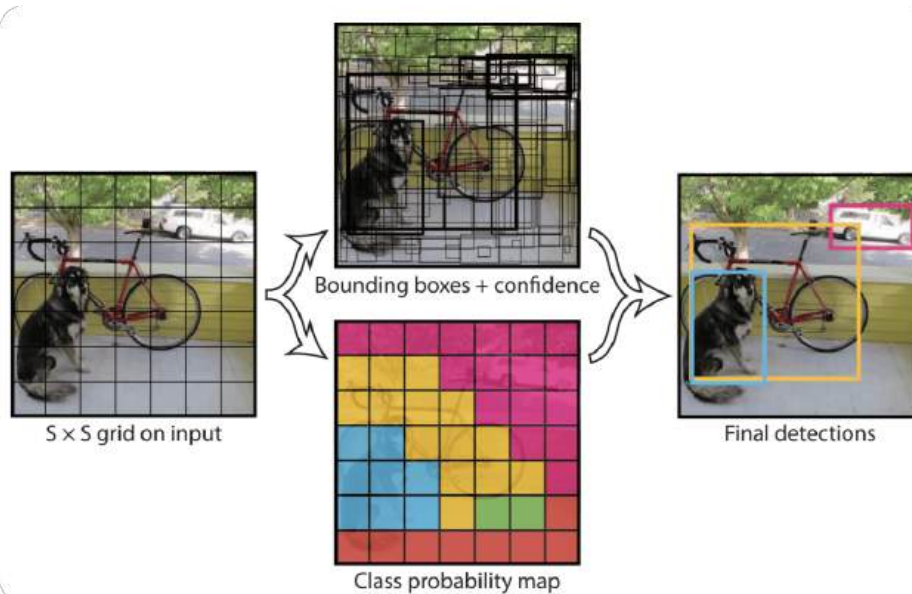


Figura 2.4: Una panoramica di come funziona YOLO.

Per il presente progetto è stata scelta una specifica versione di YOLO, la quale verrà descritta più nel dettaglio nel capitolo successivo.

2.6 Dispositivi Commerciali Esistenti

Attualmente in commercio esistono diverse soluzioni che permettono di estrarre dati da partite sportive, le quali possono essere suddivise in due macro-categorie: tracciamento fisico e analisi video. Con tracciamento fisico intendiamo tutti quei dispositivi indossabili che permettono di monitorare i parametri fisici dell'atleta (velocità, accelerazione, carichi di lavoro, frequenza cardiaca). Di seguito analizzeremo i dispositivi elencati nell'introduzione che appartengono a questa categoria:

- **FieldWiz** è un sistema EPTS (*Electronic Performance and Tracking System*) fisico. Si basa su un piccolo dispositivo (*pod*) del peso di circa 35 grammi, inserito in un'apposita tasca sulla schiena di una pettorina indossata dal giocatore. Il nucleo del sistema è un ricevitore GPS/GNSS ad alta frequenza, combinato con accelerometri, giroscopi e un sensore per la rilevazione del battito cardiaco. Durante la sessione sportiva, il *pod* registra i movimenti, i cambi di direzione, gli scatti e la distanza percorsa dall'atleta.



Figura 2.5: Dispositivo FieldWiz.

Vantaggi:

- **Altissima precisione all'aperto:** grazie al GPS a 18Hz, offre misurazioni estremamente accurate di velocità, distanze coperte e zone di campo occupate (*heat map*).
- **Integrazione biometrica:** a differenza dei sistemi basati solo su video, misura parametri fisiologici reali come lo sforzo cardiaco (ECG).
- **Soluzione di squadra ottimizzata:** la docking station permette di scaricare simultaneamente i dati di 30 giocatori in pochi minuti, rendendo il lavoro del preparatore atletico molto veloce.
- **Certificazione ufficiale:** essendo certificato FIFA, può essere indossato in gare ufficiali di calcio.

Svantaggi:

- **Limiti in ambienti chiusi (Indoor):** dipendendo dal segnale satellitare, non è in grado di rilevare dati di posizione o velocità all'interno di palazzetti (basket, pallavolo) o palestre.
 - **Assenza di dati tattico/video nativi:** restituisce esclusivamente metriche sulle performance fisiche. La correlazione con le dinamiche di gioco richiede l'incrocio manuale dei dati con i video della partita.
 - **Ingombro fisico:** l'uso della pettorina sotto la divisa da gioco può risultare fastidioso per alcuni atleti.
 - **Analisi differita:** l'analisi dei dati avviene prevalentemente post-sessione, previa esportazione su computer.
- **Talent4PLAYERS** è un sistema di tracciamento basato esclusivamente su sensori inerziali (IMU - accelerometri e giroscopi). A differenza di FieldWiz, non utilizza il GPS e si presenta come un sensore miniaturizzato da posizionare sopra la caviglia. Non calcolando la posizione assoluta del giocatore tramite satelliti, il sistema "deduce" la cinematica (corsa, passi, velocità e cambi di direzione) misurando le forze e gli impatti generati dagli arti inferiori durante il movimento. Al termine della sessione, i dati vengono trasferiti via Bluetooth allo smartphone.



Figura 2.6: Dispositivo Talent4PLAYERS.

Vantaggi:

- **Efficacia per gli sport Indoor:** l'indipendenza dal segnale GPS lo rende una soluzione eccellente per le discipline praticate al chiuso (es. pallacanestro o calcio a 5).
- **Accessibilità economica:** è concepito per ridurre i costi, essendo venduto come hardware per il singolo individuo senza l'obbligo di abbonamenti cloud costosi.
- **Comfort elevato:** posizionato alla caviglia, risulta poco invasivo e non richiede l'uso di pettorine.
- **Focalizzazione sulla biomeccanica:** rilevando i dati direttamente dagli arti inferiori, fornisce stime precise sugli impatti e sul rischio di sovraccarico fisico.

Svantaggi:

- **Assenza di posizionamento spaziale:** non disponendo di coordinate GPS, il sistema non traccia la posizione sul campo (*no heat map*), limitando fortemente l'analisi tattica.
- **Tracciamento puramente fisico:** non rileva parametri fisiologici (es. battito cardiaco) né fornisce informazioni relative alle dinamiche di gioco.
- **Nessuna trasmissione Live:** i dati non sono disponibili in tempo reale, ma consultabili unicamente al termine dell'allenamento.
- **Sensibilità al posizionamento:** un disallineamento del sensore o lo scivolamento della fascia di fissaggio possono alterare l'accuratezza dei dati raccolti.

Con analisi video intendiamo tutti quei sistemi basati su elaborazione visiva che consentono di tracciare le azioni in campo. Di seguito analizzeremo i dispositivi elencati nell'introduzione che appartengono a questa categoria:

- **SkillCorner** è una piattaforma software basata sull'intelligenza artificiale applicata alla *computer vision*, che non prevede l'impiego di dispositivi hardware fisici (sensori o telecamere proprietarie). Il sistema analizza le normali riprese televisive di una partita e utilizza algoritmi per riconoscere ogni singolo giocatore, l'arbitro e la palla. Convertendo le immagini bidimensionali in una mappa tridimensionale del campo, la piattaforma estrae dati atletici e tattici (velocità, distanze coperte, pressing, movimenti della linea difensiva) direttamente dal flusso video.



Figura 2.7: Interfaccia SkillCorner.

Vantaggi:

- **Nessun requisito hardware:** i club non necessitano dell'acquisto di sensori o dell'installazione di infrastrutture dedicate; gli atleti non devono indossare alcun dispositivo.
- **Scouting senza confini:** permette di analizzare le prestazioni fisiche e tattiche di atleti in qualsiasi parte del mondo semplicemente elaborando i video delle loro partite.
- **Integrazione tra dati tattici e fisici:** restituisce metriche avanzate (es. "corse in profondità ricevendo palla") che fondono l'atletismo con le dinamiche di gioco.

Svantaggi:

- **Dipendenza dalla regia televisiva (*Broadcast*):** l'efficacia del sistema è subordinata alle inquadrature della regia. In caso di primi piani prolungati o replay, il tracciamento dei giocatori fuori campo viene interrotto.

- **Inapplicabilità per gli allenamenti:** richiedendo riprese strutturate di tipo televisivo, non è fruibile nelle consuete sessioni di allenamento, dove spesso è presente unicamente una telecamera tattica fissa o un drone.
 - **Assenza di dati fisiologici interni:** pur calcolando accuratamente la cinematica, non fornisce indicazioni su parametri interni quali la frequenza cardiaca o il reale carico fisiologico dell'atleta.
 - **Target B2B esclusivo:** si tratta di un servizio dai costi proibitivi, rivolto unicamente a federazioni, società professionistiche e *media company*.
- **Catapult Pro Video** è un ecosistema software e cloud di video analisi di altissimo livello. Funziona come un "cervello centrale" che raccoglie i video delle partite (acquisiti da più telecamere e angolazioni) e li fonde in tempo reale con i dati provenienti dai dispositivi indossabili degli atleti (i corpetti con sensori GPS/inerziali proprietari chiamati Catapult Vector) e con le statistiche degli eventi di gioco (passaggi, tiri, falli). Questo ecosistema permette allo staff tecnico di selezionare, ad esempio, un picco di frequenza cardiaca o un'accelerazione registrata dal sensore hardware Vector e visualizzare immediatamente il video dell'azione esatta in cui l'evento è occorso. La piattaforma include inoltre strumenti di telestration (grafica in sovrimpressione) per tracciare linee tattiche e distanze direttamente sulle clip video.



Figura 2.8: Interfaccia di Catapult Pro Video.

Vantaggi:

- **Integrazione Dati-Video:** unisce in un'unica piattaforma software la video analisi con la telemetria, contestualizzando visivamente i dati fisici e biometrici raccolti dall'hardware sul campo.

- **Analisi tattica avanzata:** fornisce strumenti integrati di telestration e grafica broadcast per creare presentazioni animate e chiare, agevolando le sessioni di tattica con i giocatori.
- **Ecosistema centralizzato e collaborativo:** permette al dipartimento tecnico di elaborare, condividere e consultare i flussi di lavoro in cloud da qualsiasi dispositivo.
- **Certificazione ufficiale:** essendo certificato FIFA, è autorizzato per l'impiego in gare ufficiali di calcio.

Svantaggi:

- **Costi proibitivi:** è una soluzione *enterprise* destinata esclusivamente a club con budget elevati.
- **Dipendenza hardware ed ecosistema chiuso:** per esprimere il proprio potenziale (la sincronizzazione dati-video), il software richiede l'acquisto obbligatorio e aggiuntivo dei costosi dispositivi fisici proprietari Catapult Vector.
- **Infrastruttura esigente:** l'elaborazione di flussi video multipli in HD necessita di hardware ad alte prestazioni e di reti locali (LAN/Wi-Fi) estremamente stabili all'interno dell'impianto sportivo.
- **Elevata complessità operativa:** l'utilizzo della suite richiede competenze specifiche e un dipartimento dedicato composto da match analyst e sport scientist.
- **Infrastruttura esigente:** l'acquisizione e l'elaborazione simultanea di flussi video multipli in HD necessitano di workstation ad alte prestazioni e di reti locali (LAN/Wi-Fi) impeccabili all'interno dell'impianto sportivo.

2.7 Dispositivi Non Commerciali Esistenti

- Il **Progetto ESTECO - Politecnico di Torino**[10] è il risultato di una tesi di laurea magistrale sviluppata presso il Politecnico di Torino, inserita all'interno di un più ampio progetto di ricerca condotto dall'azienda ESTECO SpA. L'obiettivo del software è supportare gli staff tecnici nell'analisi strategica, effettuando il rilevamento e il tracciamento automatico dei giocatori e del campo all'interno di riprese televisive di partite di pallacanestro. Oltre all'individuazione degli atleti, il sistema genera una matrice di omografia per proiettare le posizioni reali dei giocatori su un modello bidimensionale del campo con visuale dall'alto (top-down), offrendo una prospettiva tattica ottimizzata.

Il funzionamento si basa sul paradigma tracking-by-detection, il quale disaccoppia la fase di rilevamento da quella di tracciamento. L'individuazione dei giocatori nei singoli frame è affidata a una Rete Neurale Convoluzionale (specificamente, l'architettura Cascade Mask R-CNN, pre-addestrata sul dataset per pedoni Caltech). Il riconoscimento del campo avviene invece con approccio semi-automatico: è richiesto l'inserimento manuale di 12 punti di riferimento nel primo frame del video; successivamente, l'algoritmo si avvale di tecniche di computer vision (quali la Trasformata di Hough e l'Optical

Flow) per seguire autonomamente le linee del campo nei fotogrammi successivi. Infine, l'associazione temporale delle rilevazioni e il calcolo delle traiettorie sono gestiti dall'algoritmo SORT, preferito a DeepSORT a causa di riscontrate problematiche di perdita del segnale in quest'ultimo.

Vantaggi:

- **Automazione dell'estrazione dati:** il sistema sostituisce l'osservazione manuale, riducendo i tempi e il carico di lavoro per gli analisti.
- **Robustezza del rilevamento:** l'architettura neurale adottata è in grado di riconoscere i giocatori anche in condizioni sfavorevoli, quali pose atipiche o parziali occlusioni causate dalla sovrapposizione tra atleti.
- **Filtraggio spaziale:** l'esatta mappatura dei confini del campo consente di escludere automaticamente gli elementi esterni al perimetro di gioco, riducendo drasticamente i falsi positivi (es. spettatori sugli spalti).

Svantaggi:

- **Efficienza computazionale limitata:** l'elaborazione richiede dai 2,8 ai 5,5 secondi per singolo fotogramma (in base alla risoluzione video), rendendo il sistema inapplicabile per l'analisi in tempo reale (live tracking) durante una partita.
 - **Imperfezioni nel tracciamento:** l'algoritmo è soggetto a perdite del bersaglio a causa della rapidità dei movimenti e della somiglianza cromatica tra le divise dei cestisti.
 - **Classificazione incompleta:** il software non discrimina in modo efficace tra giocatori e arbitri, generando falsi positivi all'interno dell'area di gioco.
 - **Dipendenza dall'inizializzazione manuale:** il sistema necessita dell'intervento umano per la calibrazione dei punti di riferimento sul fotogramma iniziale.
- Il **Progetto Koško - Università di Niš**[11] è stato ideato da un team di ricercatori e accademici afferenti alla Facoltà di Ingegneria Meccanica e alla Facoltà di Ingegneria Elettronica dell'Università di Niš, in collaborazione con Nais Robotics (Parco Scientifico e Tecnologico di Niš, Serbia). Il sistema, basato sulla computer vision, è finalizzato al rilevamento automatico dei canestri nel basket ed è stato sviluppato specificamente per applicazioni IoT (Internet of Things). Tale soluzione si integra all'interno di una piattaforma di analisi scalabile denominata "Koško", progettata per fornire statistiche in tempo reale, classifiche interattive e strumenti per il coinvolgimento degli utenti tramite un'interfaccia web. L'obiettivo primario consiste nella registrazione accurata e autonoma dei punteggi, sia in ambienti indoor che outdoor.

L'architettura si fonda su un dispositivo compatto ed economico, concepito per un'installazione diretta al di sopra del tabellone del canestro, evitando modifiche strutturali invasive. Il comparto hardware è costituito da una fotocamera ad alta risoluzione collegata a un Raspberry Pi 5, il quale gestisce l'elaborazione delle immagini localmente

(edge computing) avvalendosi dell'acceleratore hardware Hailo-8L per le reti neurali. Il processo di rilevamento si articola nelle seguenti fasi:

- **Localizzazione del canestro:** il sistema identifica in modo automatico la posizione del ferro, definendo e fissando la regione di interesse (Region of Interest o ROI).
- **Rilevamento della palla:** una Rete Neurale Convoluzionale leggera (YOLOv8n), ottimizzata per l'edge computing, si occupa del rilevamento della palla da basket all'interno dei fotogrammi.
- **Tracciamento e convalida:** le traiettorie della palla vengono monitorate e filtrate applicando criteri geometrici.
- **Conteggio dei punti:** il canestro viene convalidato esclusivamente se la palla attraversa l'area di punteggio predefinita con una traiettoria discendente e per un numero sufficiente di frame consecutivi.

Al termine della convalida del punto, i dati vengono trasmessi via Wi-Fi alla piattaforma cloud "Koško" per l'aggiornamento delle statistiche.

Vantaggi:

- **Architettura efficiente ed economica:** l'utilizzo di hardware accessibile e non invasivo, unito a un'architettura edge computing, minimizza la dipendenza da server esterni, garantendo un'elevata velocità di elaborazione e un ridotto consumo di banda.
- **Elevata specificità:** la logica di convalida, basata sul movimento discendente e sulla persistenza nei fotogrammi, riduce drasticamente i falsi positivi (raggiungendo una specificità del 96%), scartando anomalie di rimbalzo o coperture parziali del canestro.
- **Affidabilità ambientale:** il sistema dimostra di funzionare adeguatamente in ambienti esterni anche in condizioni di scarsa illuminazione (60-300 lux), con un'accuratezza complessiva dell'88%.
- **Coinvolgimento dell'utenza:** la piattaforma "Koško" incentiva l'interazione e la competizione, risultando particolarmente adatta per contesti scolastici e pubblici.

Svantaggi:

- **Tasso di rilevamento perfettibile:** il sistema presenta un recall (tasso di rilevamento dei canestri effettivi) del 76%, causato dal mancato riconoscimento di circa un canestro su quattro a causa di occlusioni, motion blur o condizioni di illuminazione non ottimali.
- **Approccio conservativo:** come evidenziato dagli autori, l'algoritmo preferisce ignorare un canestro valido pur di non registrare un falso positivo.

- **Limiti ottici:** la telecamera a frequenza standard risulta inadeguata per l'acquisizione di oggetti ad alta velocità senza sfocature, suggerendo la necessità di futuri aggiornamenti con sensori ad alto frame rate o di tipo global shutter.
- **Complessità software:** le prestazioni ottimali richiedono complessi processi di quantizzazione (precisione INT8) e conversione dei modelli (formato ONNX), che incrementano la rigidità del sistema e rendono difficoltosa l'eventuale manutenzione o sostituzione della rete neurale.
- **Il Progetto Loughborough University - Analisi delle Strategie Difensive**[12], pubblicato nel 2019 sulla rivista scientifica *Applied Sciences*, è uno studio accademico realizzato da un team di ricercatori della Loughborough University (Changjia Tian, Varuna De Silva, Michael Caine e Steve Swanson).

Il sistema utilizza tecniche di *machine learning* per il riconoscimento e la classificazione automatizzata delle strategie difensive nel gioco della pallacanestro, analizzando i dati di tracciamento spazio-temporale dei giocatori. Nello specifico, la ricerca si focalizza sulle tattiche difensive adottate per contrastare il *Pick-and-Roll* (la giocata offensiva più diffusa), identificando due specifiche reazioni difensive: lo *switch* (il cambio di marcatura) e il *trap* (il raddoppio sul portatore di palla). A differenza di molti studi precedenti, incentrati sull'interazione isolata tra l'attaccante e il suo marcatore diretto, questo approccio valuta i movimenti dell'intera squadra, analizzando il comportamento di tutti e dieci i giocatori in campo.

Il funzionamento si basa sull'elaborazione di un esteso dataset fornito da SportVU (STATS LLC), comprendente i dati di tracciamento di 32.377 possesi estratti da circa 630 partite della stagione NBA 2012-2013. L'architettura si articola in due fasi principali:

- **Assegnazione analitica:** un modello matematico stabilisce le relazioni difensive uno-contro-uno, assegnando a ciascun difensore una specifica "zona difensiva" calcolata in base alla distanza tra i giocatori, alla loro velocità e alla traiettoria di attacco verso il canestro. Monitorando costantemente tali relazioni, il sistema individua il momento esatto in cui si verifica un cambio di marcatura.
- **Classificazione data-driven:** non appena viene rilevata una variazione, si attiva un modello di classificazione che valuta undici parametri specifici (tra cui i vettori di movimento dei quattro atleti direttamente coinvolti nello scambio e le distanze dei restanti sei) per categorizzare la tattica difensiva adottata. Per questa fase sono stati testati diversi algoritmi, tra cui *K-Nearest Neighbour* (KNN), Alberi Decisionali e *Support Vector Machine* (SVM).

Vantaggi:

- **Analisi collettiva e realistica:** il sistema colma un'importante lacuna nell'analisi sportiva, tradizionalmente sbilanciata verso le metriche offensive. Analizzando i dati dell'intero gruppo squadra, riduce le interferenze esterne e offre una comprensione accurata delle dinamiche collettive.

- **Automazione su larga scala:** permette di etichettare enormi quantità di dati di tracciamento grezzi, automatizzando un compito che richiederebbe tempistiche proibitive se eseguito manualmente tramite analisi video.
- **Efficacia dell'algoritmo SVM:** tra i modelli testati, la *Support Vector Machine* si è rivelata la soluzione ottimale, raggiungendo un'accuratezza di classificazione del 68,9% e gestendo efficacemente le variabili spaziali senza il gravoso carico computazionale richiesto dal modello KNN.

Svantaggi:

- **Categorizzazione limitata:** l'approccio attuale classifica solamente due risposte difensive contro una singola giocata offensiva, non offrendo quindi una mappatura esaustiva di tutte le strategie difensive applicabili.
- **Maledizione della dimensionalità (Curse of Dimensionality):** Un numero eccessivo di feature rispetto alla quantità di dati disponibile può aumentare il rischio di overfitting e ridurre la capacità di generalizzazione.
- **Limiti del modello spaziale:** il modello analitico assegna le zone difensive basandosi esclusivamente su fattori geometrici e posizionali, trascurando parametri cruciali quali le specifiche abilità, i ruoli e l'effettiva capacità difensiva dei singoli atleti.
- **Dataset di validazione ridotto:** l'insieme di dati annotato manualmente, indispensabile per verificare l'affidabilità del sistema, risulta estremamente limitato, comprendendo solo 144 possessi a fronte degli oltre 32.000 esaminati dall'algoritmo.

Questa rassegna ha fatto emergere diverse limitazioni nelle tecnologie attuali, tra cui costi spesso proibitivi, latenze che impediscono l'analisi in tempo reale durante gli allenamenti, fastidio fisico legato all'hardware indossabile e forte dipendenza da infrastrutture di rete complesse o riprese di tipo broadcast televisivo.

Alla luce di tali criticità, la direzione intrapresa per la soluzione proposta in questa tesi punta alla realizzazione di un ecosistema di analisi video economico, compatto e del tutto autonomo. L'obiettivo è sfruttare il paradigma dell'edge computing integrando un hardware accessibile, come il Raspberry Pi 5 e l'acceleratore AI HAT+, con un algoritmo di object detection leggero ed efficiente come YOLOv8n. Questo approccio permetterà di elaborare i fotogrammi direttamente a bordo campo con bassissima latenza e di azionare simultaneamente un sistema di servomotori per orientare l'inquadratura, fornendo a staff e giocatori un tracciamento in tempo reale senza dover dipendere da operatori umani, server in cloud o equipaggiamenti dai costi inaccessibili.

Capitolo 3

Metodologia

Questo capitolo illustra nel dettaglio la metodologia adottata per la realizzazione del sistema di tracciamento automatizzato, descrivendo le specifiche tecniche e le scelte progettuali alla base dell'intero ecosistema. La trattazione è suddivisa in due macro-aree principali: la prima parte è dedicata alla metodologia hardware, all'interno della quale verranno analizzate le caratteristiche dei singoli componenti fisici impiegati, tra cui il Raspberry Pi 5, il modulo AI HAT+ per l'accelerazione neurale, la telecamera IMX219 e i servomotori necessari all'orientamento dell'inquadratura. La seconda parte si concentra invece sulla metodologia software, esplorando le ragioni dietro la scelta del modello predittivo YOLOv8n e analizzando in modo approfondito il funzionamento dei moduli di codice sviluppati per garantire l'elaborazione video in tempo reale e il controllo simultaneo dei rotori.

Nota: Nel presente lavoro il termine tracciamento indica il controllo dinamico dell'inquadratura sulla base delle rilevazioni frame-by-frame e non un algoritmo di multi-object tracking con associazione persistente delle identità.

3.1 Architettura e flusso dei dati

In questa sezione verrà illustrato il ruolo dei vari componenti, il flusso delle informazioni e l'ordine di scambio dei dati, delineando il processo che trasforma l'informazione grezza nel risultato desiderato.

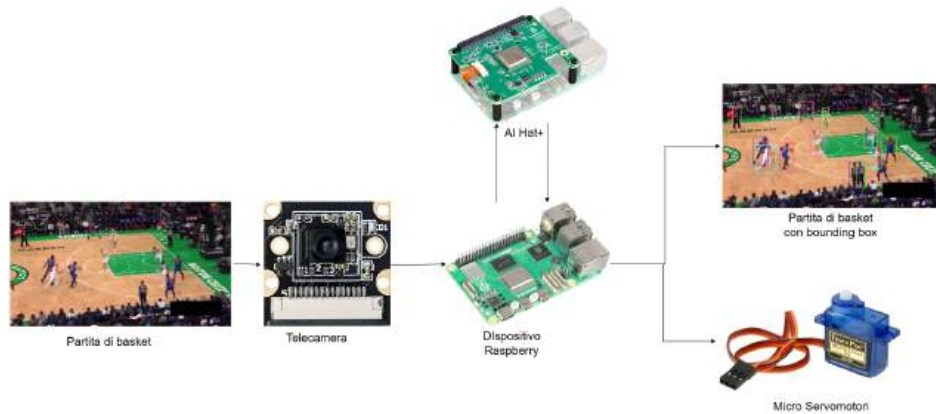


Figura 3.1: Schema a blocchi dell'architettura del sistema.

Il processo ha inizio con l'acquisizione del flusso video da parte della **telecamera**, la quale trasmette i dati al **Raspberry Pi**. Quest'ultimo elabora preliminarmente il *frame* ricevuto per renderlo compatibile e lo inoltra all'**AI HAT**. L'acceleratore hardware, tramite un modello di *object detection*, analizza il fotogramma, rileva la posizione dei giocatori al suo interno e restituisce le coordinate spaziali al **Raspberry**. Una volta ricevute tali informazioni, il microcomputer traccia le *bounding box* intorno ai soggetti riconosciuti e restituisce a schermo il fotogramma elaborato. Parallelamente, a seconda della posizione dei giocatori individuati, il sistema invia appositi segnali di controllo ai **micro servomotori**, i quali orientano meccanicamente l'impianto per mantenere costantemente l'azione all'interno del campo visivo.

3.2 Metodologia Hardware

Di seguito verranno descritte in dettaglio le caratteristiche dei componenti hardware utilizzati per la realizzazione del progetto.

3.2.1 Raspberry Pi 5

Dotato di un processore Arm Cortex-A76 quad-core a 64 bit operante a 2.4 GHz, il Raspberry Pi 5 offre un incremento di prestazioni della CPU pari a 2-3 volte rispetto al suo predecessore, il Raspberry Pi 4. Grazie a una GPU VideoCore VII da 800 MHz, il dispositivo garantisce un sostanziale miglioramento delle prestazioni grafiche, supportando una doppia uscita display 4Kp60 tramite HDMI. Il processore del segnale d'immagine (ISP), riprogettato internamente, fornisce il supporto per telecamere all'avanguardia, assicurando un'esperienza desktop fluida e aprendo le porte a nuove applicazioni.

Per la prima volta, si tratta di un computer Raspberry Pi di dimensioni standard che impiega silicio sviluppato internamente. Il "southbridge" RP1 fornisce la maggior parte delle capacità di I/O, offrendo un salto qualitativo nelle prestazioni e nelle funzionalità delle periferiche. La larghezza di banda USB aggregata è più che raddoppiata, garantendo velocità di trasferimento superiori verso unità esterne; inoltre, le interfacce per fotocamera e display

MIPI a due corsie dei modelli precedenti sono state sostituite da una coppia di transceiver MIPI a quattro corsie da 1.5 Gbps, triplicando la larghezza di banda totale e supportando qualsiasi combinazione fino a due telecamere o display. Le prestazioni massime della scheda SD sono raddoppiate grazie al supporto per la modalità SDR104 ad alta velocità e, per la prima volta, la piattaforma espone un'interfaccia PCI Express 2.0 a corsia singola per il supporto di periferiche ad alta larghezza di banda.



Figura 3.2: Dispositivo Raspberry Pi 5 16Gb.

Specifiche Processore: CPU Broadcom BCM2712 quad-core a 64 bit Arm Cortex-A76 a 2.4 GHz, con estensioni crittografiche, cache L2 da 512 KB per core e cache L3 condivisa da 2 MB.

Caratteristiche principali:

- GPU VideoCore VII, con supporto per OpenGL ES 3.1, Vulkan 1.3.
- Doppia uscita display HDMI® 4Kp60 con supporto HDR.
- Decoder HEVC 4Kp60.
- SDRAM LPDDR4X-4267 da 16 GB.
- Wi-Fi® 802.11ac dual-band.
- Bluetooth 5.0 / Bluetooth Low Energy (BLE).
- Slot per schede microSD, con supporto per la modalità SDR104 ad alta velocità.
- 2 porte USB 3.0, con supporto per il funzionamento simultaneo a 5 Gbps.
- 2 porte USB 2.0.
- Gigabit Ethernet, con supporto PoE+ (richiede HAT PoE+ separato).
- 2 ricetrasmittitori MIPI a 4 corsie per fotocamera/display.

- Interfaccia PCIe 2.0 x1 per periferiche veloci (richiede HAT M.2 separato o altro adattatore).
- Alimentazione CC 5V/5A tramite USB-C, con supporto Power Delivery.
- Header standard Raspberry Pi a 40 pin.
- Real-time clock (RTC), alimentato da batteria esterna.
- Pulsante di accensione.

Motivazione della scelta: La scelta del Raspberry Pi 5 è dettata dalla necessità di bilanciare prestazioni elevate e minimizzazione dei costi. Rispetto ad altre soluzioni di calcolo o ai PC tradizionali, questa scheda offre la potenza computazionale necessaria per gestire l'elaborazione locale (edge computing) a una frazione del prezzo, mantenendo un fattore di forma compatto ideale per un'installazione agevole a bordo campo.

3.2.2 AI HAT+

L'AI HAT+ è una scheda di espansione per Raspberry Pi 5 dotata di un chip acceleratore AI integrato, ovvero la Neural Processing Unit (NPU) Hailo. La NPU Hailo consente al Raspberry Pi 5 di eseguire localmente modelli di intelligenza artificiale con accelerazione hardware, eliminando la necessità di inviare dati a un server cloud remoto per l'elaborazione. L'elaborazione locale riduce l'esposizione dei dati verso servizi remoti ed elimina la componente di latenza dovuta alla comunicazione con il cloud.

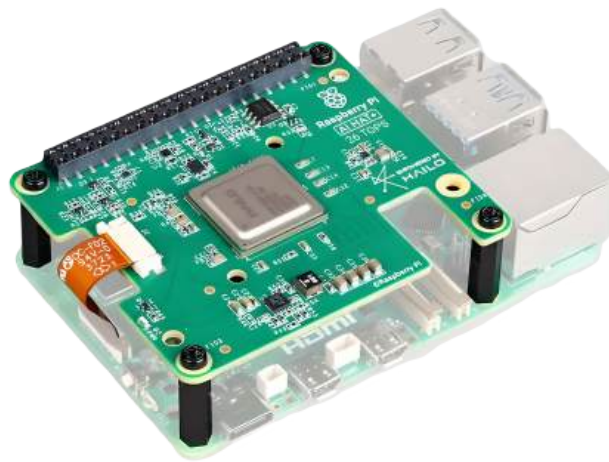


Figura 3.3: Dispositivo AI HAT+.

Caratteristiche:

- Acceleratore Hailo-8 che offre prestazioni di inferenza di 26 TOPS.

- Completa integrazione nello stack software della fotocamera del Raspberry Pi.
- Conforme alla specifica Raspberry Pi HAT+.
- Fornito con header di impilamento da 16 mm, distanziatori e viti per consentire il montaggio su Raspberry Pi 5 con Raspberry Pi Active Cooler installato.
- Temperatura operativa da 0°C a 50°C (ambiente).

Feature	AI HAT+
Accelerator chip (Hailo NPU)	Hailo-8 (26 TOPS, INT8)
Memory (RAM)	Utilizza la memoria del Raspberry Pi 5
Large Language Model (LLM) support	Non supportato
Vision-Language Model (VLM) support	Non supportato
Use cases	Object detection, post-elaborazione fotocamera, robotica, carichi di lavoro neurali moderati

Tabella 3.1: Specifiche tecniche AI HAT+.

Motivazione della scelta: L'impiego dell'AI HAT+ rappresenta il compromesso ideale per abilitare l'inferenza di reti neurali in tempo reale su un sistema embedded economico. Affidare l'elaborazione alla sola CPU avrebbe causato ritardi inaccettabili, mentre l'uso di GPU esterne o server cloud avrebbe fatto lievitare vertiginosamente i costi. Questo modulo garantisce l'alta potenza di calcolo richiesta mantenendo la spesa estremamente contenuta e riducendo la latenza dovuta alla comunicazione con servizi remoti.

3.2.3 Raspberry Pi 5 Active Cooler

Il Raspberry Pi 5 Active Cooler è stato impiegato per mantenere la temperatura del processore entro un range operativo accettabile. Prima della sua installazione, infatti, il surriscaldamento causava fenomeni di thermal throttling, portando a una diminuzione delle prestazioni del dispositivo.

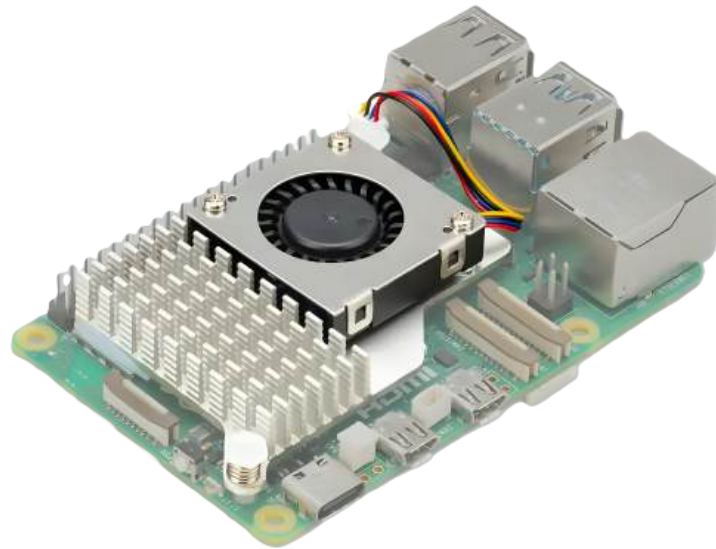


Figura 3.4: Dispositivo di raffreddamento.

Caratteristiche:

- Dissipatore di calore monoblocco in alluminio anodizzato.
- Ventola termocontrollata montata sul dissipatore e fissata con tre viti.
- Perni a pressione con molla per il montaggio su Raspberry Pi 5.
- Pad termici pre-applicati per il trasferimento del calore.

Specifiche tecniche:

- **Tensione di ingresso:** 5V CC fornita tramite connettore per ventola a quattro pin su Raspberry Pi 5.
- **Controllo della velocità della ventola:** Modulazione di larghezza di impulso (PWM) con tachimetro.
- **Flusso d'aria massimo:** 1,09 CFM.
- **Velocità massima della ventola:** 8000 RPM +/- 15%.
- **Materiale del prodotto:** Alluminio anodizzato.

Motivazione della scelta: Poiché l'elaborazione video intensiva e l'uso dell'acceleratore AI generano un notevole carico termico, si rischiavano vistosi cali di framerate a causa del throttling. L'Active Cooler, essendo un componente dal costo irrisorio, previene efficacemente questo problema e assicura che il sistema mantenga le prestazioni massime e costanti che sono vitali per l'analisi e il tracciamento in tempo reale.

3.2.4 Micro servomotore SG90 9g

Si tratta di un servomotore compatto e leggero che fornisce un controllo angolare preciso, risultando ideale per la robotica, i veicoli telecomandati e i sistemi di automazione. Per le finalità del progetto ne sono stati impiegati due: uno dedicato alla rotazione orizzontale della telecamera e uno per l'inclinazione verticale.

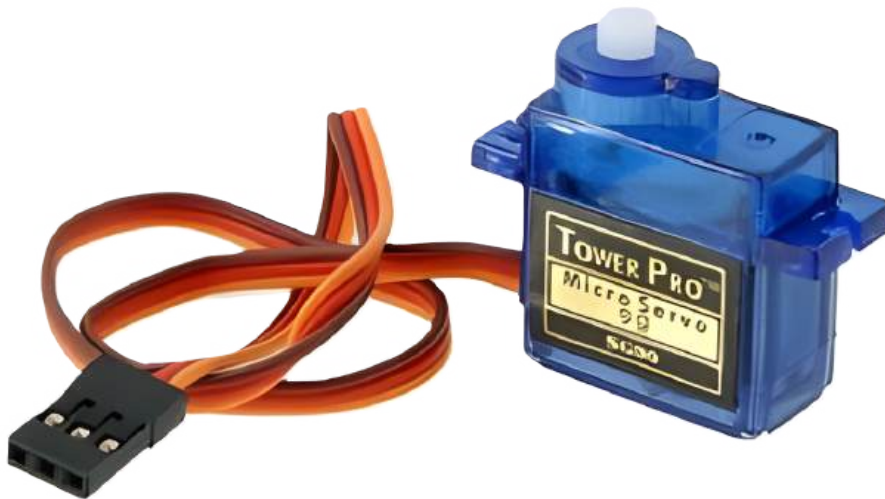


Figura 3.5: Servo motore.

Specifiche tecniche principali:

- **Tensione operativa:** 3.0 V – 6.0 V (sono consigliati 4.8 V – 5.0 V per coppia e stabilità ottimali).
- **Coppia di stallo:** 1.8 kg·cm (a 4.8 V). Al di sotto di 4.5 V la coppia diminuisce significativamente.
- **Velocità operativa:** 0.10 secondi/60° (a 4.8 V).
- **Intervallo di rotazione:** 0° – 180° (gli arresti meccanici limitano la rotazione, non forzare oltre l'intervallo).
- **Larghezza della banda morta:** 5 μ s (variazione minima dell'ampiezza dell'impulso per avviare il movimento).
- **Peso:** 9 g (± 1 g), inclusi cavi e connettore.
- **Dimensioni:** 22.8 x 12.2 x 27.4 mm.

- **Tipo di connettore:** Basetta femmina a 3 pin (standard JR/Futaba). Ordine dei pin: Segnale (S), Alimentazione (VCC), Terra (GND).
- **Materiale dell'ingranaggio:** Nylon (non adatto per applicazioni a coppia elevata o a carico continuo).

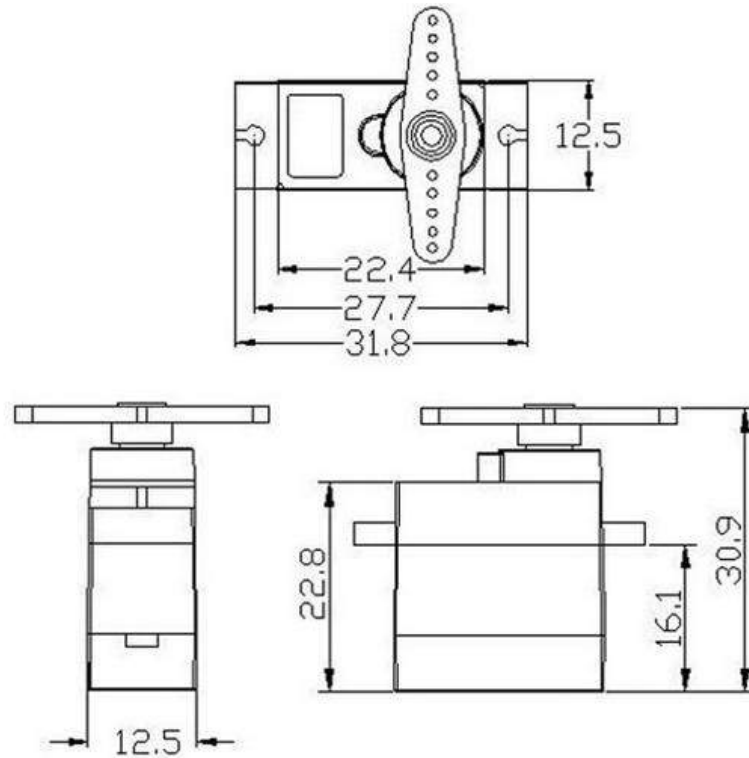


Figura 3.6: Dimensioni servo motore.

Istruzioni per il Cablaggio:

- **Filo marrone o nero (Terra/GND):** da collegare alla terra del sistema.
- **Filo rosso (Alimentazione/VCC):** da collegare a un'alimentazione stabile da 4.8 V – 5.0 V.
- **Filo arancione o giallo (Segnale/PWM):** da collegare a un pin del microcontrollore compatibile con segnali PWM.

Motivazione della scelta: L'impiego dell'SG90 massimizza l'efficienza economica dell'infrastruttura hardware. Essendo il modulo telecamera molto leggero, non era necessario investire in costosi motori ad alta coppia. L'SG90 fornisce la velocità sufficiente per la prototipazione e per movimenti moderati, pur rappresentando un possibile limite nelle azioni più rapide.

3.2.5 Modulo Camera FNK0056B IMX219

Il modulo camera FNK0056B è un componente hardware prodotto da Freenove, progettato specificamente per l'integrazione in progetti di elettronica, robotica e visione artificiale.

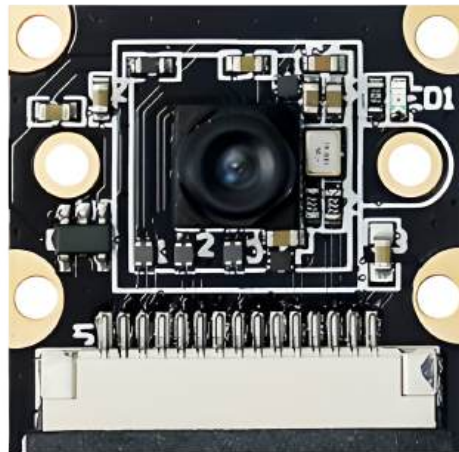


Figura 3.7: Modulo Camera IMX219.

Specifiche tecniche:

- **Risoluzione foto:** 8 megapixel.
- **Risoluzione sensore:** 3280 x 2464 pixel.
- **Dimensione sensore:** 1/4 di pollice.
- **Dimensione pixel:** $1,12 \times 1,12 \mu\text{m}$.
- **Obiettivo:** a fuoco fisso.
- **Campo visivo:** 120° .
- **Apertura:** F/2.2.
- **Modalità video:** 1080p 30fps, 720p 60fps.

Motivazione della scelta: Questa telecamera offre un eccellente connubio tra prestazioni visive ed economia hardware. La connessione diretta CSI-2 elimina i colli di bottiglia tipici delle periferiche USB. Inoltre, il supporto per framerate elevati (720p a 60fps) permette di acquisire sequenze fluide riducendo al minimo le sfocature (motion blur) tipiche degli sport dinamici, fornendo all'intelligenza artificiale fotogrammi chiari e adatti all'elaborazione con una spesa trascurabile rispetto all'equipaggiamento professionale.

Di seguito sono riportate le istruzioni per la corretta configurazione e il funzionamento della telecamera sul Raspberry Pi 5:

1. Apri il file config.txt.

```
sudo nano /boot/firmware/config.txt
```

2. Disabilitare la funzione di rilevamento automatico della telecamera.

```
camera_auto_detect=0
```

3. Aggiungi la seguente istruzione in fondo.

```
dtoverlay=imx219,cam0
```

4. Salva il file e riavvia il Raspberry Pi.

```
sudo reboot
```

3.3 Metodologia Software

Di seguito vengono analizzati i moduli software e le librerie implementate per il corretto funzionamento del sistema, partendo dall'elaborazione video fino ad arrivare al controllo dei motori.

3.3.1 Modello YOLOv8n

YOLOv8 è stato rilasciato da Ultralytics il 10 gennaio 2023, offrendo prestazioni all'avanguardia in termini di precisione e velocità[13]. Basandosi sui progressi delle versioni precedenti, YOLOv8 ha introdotto nuove funzionalità e ottimizzazioni che lo rendono una scelta ideale per diverse attività di object detection in una vasta gamma di applicazioni.

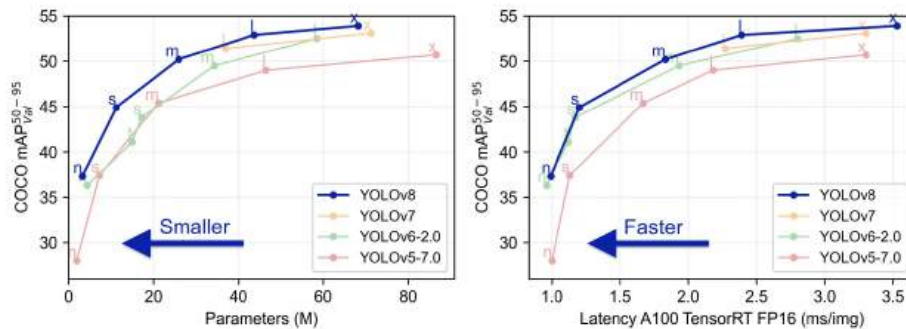


Figura 3.8: Analisi comparativa.

Nel grafico precedente è presentata un'analisi comparativa delle prestazioni tra diverse generazioni dell'architettura YOLO (YOLOv8, YOLOv7, YOLOv6-2.0 e YOLOv5-7.0) valutate sul dataset COCO. Il confronto evidenzia i trade-off tra l'accuratezza del modello, misurata tramite la metrica mAP₅₀₋₉₅, e la sua efficienza sia in termini di dimensioni che di velocità di esecuzione. I risultati si articolano in due grafici principali:

- **Accuratezza rispetto alla Dimensione del Modello (Pannello di Sinistra):** il grafico mette in relazione l'accuratezza (mAP) con la complessità strutturale della rete, espressa in milioni di parametri. La curva di YOLOv8 (in blu scuro) si posiziona stabilmente al di sopra di tutte le versioni precedenti. Questo andamento dimostra che, per ogni specifica taglia del modello (dalle varianti nano "n" alle varianti extra-large "x"), YOLOv8 raggiunge un'accuratezza superiore a parità di numero di parametri, risultando un'architettura efficiente in termini di allocazione della memoria.
- **Accuratezza rispetto alla Velocità di Inferenza (Pannello di Destra):** il secondo grafico valuta l'accuratezza in funzione della latenza di inferenza, misurata in millisecondi per immagine (ms/img). Anche in questo scenario, YOLOv8 mostra un compromesso favorevole mostrando che a parità di velocità di elaborazione restituisce una precisione più elevata, oppure, a parità di accuratezza target, richiede tempi di inferenza significativamente inferiori.

In conclusione, l'analisi comparativa attesta che YOLOv8 costituisce la soluzione più idonea rispetto alle architetture precedenti (YOLOv5, YOLOv6, YOLOv7). Tale modello è stato pertanto selezionato poiché garantisce un bilanciamento ottimale tra accuratezza predittiva e onere computazionale per lo specifico contesto di utilizzo.

Caratteristiche principali di YOLOv8

- **Architetture avanzate di backbone e neck:** YOLOv8 impiega architetture all'avanguardia, che garantiscono prestazioni ottimali nell'estrazione delle feature e nell'object detection.
- **Ultralytics Head split senza ancoraggi:** adotta un approccio anchor-free che contribuisce a una migliore precisione e a un processo di rilevamento più efficiente rispetto agli approcci basati su anchor box.
- **Compromesso ottimizzato tra precisione e velocità:** mantenendo un equilibrio ottimale, YOLOv8 è adatto per attività di object detection in tempo reale in diversi settori applicativi.
- **Varietà di modelli preaddestrati:** offre una gamma di modelli pre-addestrati per soddisfare varie attività e requisiti prestazionali, facilitando l'individuazione della soluzione ideale per lo specifico caso d'uso.

La tabella seguente illustra le metriche prestazionali delle cinque varianti del modello YOLOv8 ordinate per complessità crescente: Nano (n), Small (s), Medium (m), Large (l) ed Extra-Large (x).

L'adozione di YOLOv8n per sistemi embedded

Un Raspberry Pi è un sistema embedded caratterizzato da risorse computazionali ridotte, alimentazione limitata e assenza di una GPU dedicata ad alte prestazioni. L'inferenza viene quindi eseguita primariamente tramite CPU ARM o moduli edge dedicati. La scelta della versione Nano (YOLOv8n) si giustifica per i seguenti motivi:

Modello	dimensione (pixel)	mAP ^{val} 50–95	Velocità CPU ONNX (ms)	Velocità A100 TensorRT (ms)	params (M)	FLOPs (GFLOPs)
YOLOv8n	640	37.3	80.4	0.99	3.2	8.7
YOLOv8s	640	44.9	128.4	1.20	11.2	28.6
YOLOv8m	640	50.2	234.7	1.83	25.9	78.9
YOLOv8l	640	52.9	375.2	2.39	43.7	165.2
YOLOv8x	640	53.9	479.1	3.53	68.2	257.8

Tabella 3.2: Prestazioni dei modelli YOLOv8.

- **Costo computazionale minimo (FLOPs ridotti):** YOLOv8n richiede soltanto 8.7 GFLOPs, a fronte dei 28.6 GFLOPs della versione Small e degli oltre 257.8 GFLOPs della versione Extra-Large. Sulla CPU di un Raspberry Pi, un carico computazionale superiore comporterebbe latenze di diversi secondi per fotogramma, rendendo impossibile l’elaborazione fluida o in tempo reale.
- **Impronta di memoria e overhead minimi (3.2M di parametri):** il file dei pesi di YOLOv8n occupa circa 6 MB, richiedendo un’allocazione minima nella memoria RAM. Ciò riduce la latenza di caricamento, previene problemi di out-of-memory e preserva memoria per il resto del sistema operativo e della pipeline software.
- **Trade-off ottimale tra reattività e accuratezza:** sebbene YOLOv8n abbia un’accuratezza nominale inferiore rispetto ai modelli più pesanti (mAP = 37.3 contro mAP = 53.9 di YOLOv8x), offre un compromesso favorevole tra accuratezza e costo computazionale per applicazioni edge, garantendo il più alto throughput in termini di frame per secondo (FPS).

Nello specifico, per la realizzazione del presente progetto è stato impiegato il modello YOLOv8n con estensione `.hef`, al fine di sfruttare l’accelerazione hardware fornita dalla NPU dell’AI HAT+.

3.3.2 Modulo per l’object detection

Lo script Python è progettato per operare su un Raspberry Pi 5 equipaggiato con un AI HAT+ (modulo NPU Hailo-8). Di seguito viene fornita un’analisi dettagliata del suo funzionamento, suddivisa per punti principali:

Funzionamento del Codice

Lo script esegue due task principali in modo concorrente (tramite l’utilizzo di thread):

- **Elaborazione Video (object detection):** acquisisce un file video e impiega l’acceleratore AI Hailo-8 per eseguire il rilevamento degli oggetti in tempo reale tramite il modello YOLOv8.
- **Monitoraggio Termico:** parallelamente all’elaborazione visiva, rileva continuamente la temperatura sia del processore principale del Raspberry Pi 5 (SoC) sia del chip Hailo-8.

L'output finale consiste in una finestra video che mostra il filmato con le bounding box degli oggetti rilevati dall'intelligenza artificiale. In alto a sinistra viene generato un pannello in sovrapposizione (overlay) che visualizza:

- L'ora attuale.
- La temperatura del Raspberry Pi (con un indicatore cromatico verde/giallo/rosso).
- La temperatura del chip Hailo-8.
- Gli FPS (Frame Per Second) correnti.



Figura 3.9: Finestra video con bounding box e HUD che mostra temperatura del dispositivo e FPS.

Librerie Principali Utilizzate

- **cv2 (OpenCV):** costituisce il nucleo della gestione video. Viene impiegata per catturare i fotogrammi tramite GStreamer, sovrapporre elementi grafici come il pannello delle temperature e gli FPS (`cv2.putText`, `cv2.rectangle`) e renderizzare la finestra a schermo (`cv2.imshow`).
- **hailo_platform (VDevice):** è la libreria ufficiale fornita da Hailo. Nello script è utilizzata per comunicare a basso livello con il chip Hailo-8 e leggere i dati dei sensori di temperatura interni (`ts0` e `ts1`), gestendo l'accesso condiviso per non interferire con l'elaborazione video.

- **threading:** consente di eseguire il monitoraggio termico in background. In tal modo, l'aggiornamento asincrono delle temperature non compromette la fluidità del flusso video.
- **Librerie di sistema (os, time, datetime):** utilizzate per leggere la temperatura del dispositivo dai file di sistema (es. `/sys/class/thermal/...`), calcolare gli FPS in base al tempo trascorso e gestire le variabili d'ambiente.

Pipeline GStreamer

GStreamer è un framework multimediale ad alte prestazioni[14]. Al fine di evitare i rallentamenti derivanti da una decodifica video gestita interamente in Python, lo script delega il carico computazionale a GStreamer definendo una "pipeline" (un canale di elaborazione sequenziale dei dati).

Di seguito viene analizzata la composizione della pipeline:

```
pipeline = (
    f"filesrc location={VIDEO_PATH} ! decodebin ! "
    f"queue max-size-buffers=3 ! videoconvert ! videoscale ! "
    f"video/x-raw,format=RGB,pixel-aspect-ratio=1/1 ! "
    f"queue max-size-buffers=3 ! hailonet hef-path={HEF_PATH} scheduling-algorithm=1 ! "
    f"queue max-size-buffers=3 ! hailofilter so-path={POST_PROCESS_SO} qos=false ! "
    f"hailooverlay ! videoconvert ! video/x-raw,format=BGR ! "
    f"appsink sync=false max-buffers=1 drop=true"
)
```

Figura 3.10: Pipeline GStreamer.

- **Lettura e Decodifica:** `filesrc ! decodebin` prende il file video `test_video.mp4` e lo decodifica in fotogrammi non compressi.
- **Preparazione:** `videoconvert ! videoscale` adatta lo spazio colore (RGB) e le dimensioni delle immagini per renderle compatibili con gli input richiesti dalla rete neurale.
- **Inferenza AI (hailonet):** rappresenta il plugin centrale. Carica il modello YOLOv8 compilato (`yolov8n.hef`) e lo esegue fisicamente sull'acceleratore hardware Hailo-8. L'impostazione del parametro `scheduling-algorithm=1` (Round Robin) è fondamentale, poiché consente allo script Python di accedere al chip simultaneamente per la lettura delle temperature, prevenendo instabilità di sistema.
- **Post-elaborazione (hailofilter):** l'output grezzo della rete neurale (valori numerici e probabilità) viene processato da una libreria C++ (`libyolo_hailortpp_post.so`), che lo converte nelle coordinate spaziali per le bounding box.
- **Disegno (hailooverlay):** traccia materialmente i rettangoli e le etichette di classificazione (es. "persona", "automobile") direttamente sui fotogrammi a livello hardware.
- **Uscita verso OpenCV (appsink):** consegna il fotogramma elaborato (completo di bounding box) allo script Python. Successivamente, OpenCV sovrappone il pannello grafico delle temperature e mostra il risultato a schermo.

L'impiego della pipeline permette di delegare a GStreamer la gestione della pipeline multimediale e alla NPU Hailo-8 l'inferenza neurale, fornendo all'ambiente Python un output quasi definitivo su cui applicare esclusivamente le interfacce grafiche finali.

3.3.3 Modulo Gestione Telecamera

Il presente modulo è preposto alla visualizzazione in tempo reale del flusso video proveniente dalla telecamera **IMX219** (connessa tramite interfaccia CSI-2) sul Raspberry Pi 5. Il software implementa funzionalità avanzate rispetto a un semplice script di visualizzazione:

- **Diagnostica hardware:** prima dell'apertura della finestra video, interroga il bus I2C per verificare il collegamento fisico della telecamera e richiede conferma ai driver di sistema (`libcamera`).
- **Gestione fallback automatico:** per mitigare i noti problemi del Raspberry Pi 5 nella negoziazione del formato video, lo script itera su diverse configurazioni sicure (es. NV12, BGRA) fino a individuare quella ottimale per evitare instabilità di sistema.
- **Heads-Up Display (HUD):** sovrappone al video un testo nativo contenente l'ora, la temperatura del processore, gli FPS reali e la risoluzione.

Librerie principali utilizzate

- `gi.repository.Gst` e `gi.repository.GLib` (**GStreamer Python Bindings**): a differenza dell'approccio basato su OpenCV, questo script impiega le librerie native di GStreamer per Python. Forniscono un approccio asincrono ad alte prestazioni basato sul motore di eventi `GLib.MainLoop()`, che gestisce il video, le eccezioni e i timer in modo asincrono, senza bloccare l'esecuzione.
- `subprocess`: impiegata nella fase iniziale per eseguire comandi terminale Linux (es. `i2cdetect` e `libcamera-hello --list-cameras`) e analizzarne l'output per verificare il corretto riconoscimento hardware della telecamera da parte dei driver.
- `threading` e `time`: gestiscono il conteggio in sicurezza dei fotogrammi (FPS), evitando conflitti di esecuzione tra il flusso video ad alta velocità e i cicli di lettura dei dati termici (impostati ogni 2 secondi).

Logica di Funzionamento

L'approccio implementato si articola in tre fasi principali:

Fase A: Scoperta e Negoziazione (Fallback Automatico)

Lo script testa un elenco di pipeline candidate impostandole nello stato "PAUSED" (attivazione dei sensori senza avvio del flusso video). Se il bus restituisce un errore di incompatibilità del formato video, la pipeline viene distrutta e si procede con la successiva configurazione candidata.

Fase B: La Pipeline di Streaming

Individuata la combinazione ottimale, viene avviata la pipeline operativa di base.

```
def _get_pipeline_candidates() -> list[tuple[str, str]]:
    """Restituisce una lista di pipeline candidate (nome, stringa)."""
    hud_element = (
        'textoverlay name=hud valignment=top halignment=left '
        'font-desc="monospace Bold 14" shaded-background=true text="Avvio..." '
    )
    return [
        # 1. Formato NV12 forzato + doppio videoconvert
        (
            "NV12 + HUD",
            f"libcamerasrc ! video/x-raw,format=NV12 ! "
            f"videoconvert ! {hud_element}! "
            f"videoconvert ! autovideosink sync=false name=sink"
        ),
    ],
```

Figura 3.11: Codice della Pipeline video.

Componenti della pipeline:

- **libcamerasrc**: costituisce la sorgente. Dialoga con i driver ufficiali Raspberry Pi (**libcamera**) per acquisire i fotogrammi grezzi dal sensore IMX219 tramite il connettore hardware CSI-2.
- **video/x-raw,format=...**: impone un formato video specifico e sicuro (es. NV12) per prevenire anomalie e crash.
- **textoverlay name=hud**: inietta stringhe di testo direttamente nel flusso GStreamer (assegnando il nome *hud* per successive modifiche dinamiche via codice Python), sostituendo le funzioni di disegno standard di OpenCV.
- **autovideosink**: acquisisce i fotogrammi finali e apre automaticamente la finestra di visualizzazione a schermo.

Fase C: Il "Pad Probe" e l'HUD Dinamico

Per il calcolo degli FPS e l'aggiornamento del testo, GStreamer consente di agganciare delle sonde (Pad Probe) ai collegamenti virtuali tra i componenti della pipeline. La funzione `on_frame_probe` funge da sonda: al passaggio di ogni singolo fotogramma, si innesca incrementando un contatore.

Parallelamente, un timer asincrono (`GLib.timeout_add_seconds`) si attiva ogni due secondi per calcolare matematicamente gli FPS reali basandosi sul contatore, leggere la temperatura del dispositivo e iniettare la stringa di testo aggiornata all'interno della proprietà `text` del componente `textoverlay`. Questo meccanismo garantisce un aggiornamento fluido dell'HUD in tempo reale senza la necessità di estrarre e processare i fotogrammi nell'ambiente Python.

```

_fps_lock      = threading.Lock()
_fps_frame_count = 0
_fps_last_count = 0
_fps_last_time  = 0.0

def _on_frame_probe(pad, info) -> Gst.PadProbeReturn:
    """Callback GStreamer pad probe per contare i frame."""
    global _fps_frame_count
    with _fps_lock:
        _fps_frame_count += 1
    return Gst.PadProbeReturn.OK

```

Figura 3.12: Porzione di codice che mostra come vengono contati gli FPS.

3.3.4 Modulo Gestione Rotori

Il presente modulo gestisce il controllo manuale dei due servomotori (assegnati all'asse X e all'asse Y) collegati al Raspberry Pi 5, tramite l'input da tastiera. Il software genera un'interfaccia testuale nel terminale (Terminal User Interface o TUI) che consente all'utente di:

- Movimentare i motori lungo gli assi orizzontale e verticale mediante le frecce direzionali.
- Ripristinare istantaneamente la posizione centrale tramite la pressione del tasto 'c'.
- Terminare l'esecuzione del programma in sicurezza mediante il tasto 'q'.

Librerie principali utilizzate

- **rpi_hardware_pwm (HardwarePWM)[15]**: risulta fondamentale per la gestione fisica dei rotori, in quanto consente di generare un segnale PWM (Pulse Width Modulation) basandosi direttamente sull'hardware del dispositivo (nello specifico, il chip RP1 del Raspberry Pi 5). Tale approccio elimina i micro-ritardi (jitter) tipici del PWM software, garantendo un segnale stabile e prevenendo vibrazioni anomale dei motori.
- **curses**: impiegata per la creazione dell'interfaccia a riga di comando. Permette di aggiornare il testo in punti specifici dello schermo e di intercettare gli eventi della tastiera in tempo reale e in modalità non bloccante (nodelay), omettendo la necessità di confermare l'input con il tasto "Invio".
- **time**: utilizzata per richiamare la funzione `time.sleep(0.01)`, la quale inserisce una pausa di 10 millisecondi nel ciclo di esecuzione al fine di ottimizzare il carico computazionale, evitando che lo script saturi l'utilizzo della CPU durante l'attesa degli input.

Logica di Funzionamento

Il sistema si basa su un ciclo di controllo (Control Loop) strutturato nelle seguenti fasi:

- **Inizializzazione Motori:** il rotore Y viene assegnato al canale PWM 0 (Pin 12) e il rotore X al canale PWM 1 (Pin 13). La frequenza viene impostata a 50 Hz (standard per i servomotori analogici, equivalente a un impulso ogni 20 millisecondi), con un posizionamento iniziale centrato corrispondente a un Duty Cycle del 7.5%.
- **Ciclo di esecuzione:** tramite un costrutto `while True`, il sistema interroga ciclicamente la tastiera avvalendosi della funzione `stdscr.getch()`.
- **Elaborazione dell'input e gestione dei limiti:** la pressione delle frecce direzionali modifica le variabili spaziali logiche (`pos_x` e `pos_y`), le quali variano tipicamente in un intervallo compreso tra -1.0 (rotazione minima) e +1.0 (rotazione massima). La variazione avviene per incrementi discreti costanti (definiti da una variabile `STEP` pari a 0.03) per garantire un movimento fluido. Apposite funzioni di vincolo (`min()` e `max()`) prevengono il superamento dei limiti meccanici (ad esempio, imponendo un blocco a 0.5 sull'asse Y per evitare forzature strutturali).
- **Mappatura Hardware (La Traslazione):** i valori spaziali astratti vengono convertiti e inviati alla funzione di posizionamento hardware. Poiché i servomotori interpretano esclusivamente la durata dell'impulso elettrico espressa in percentuale (Duty Cycle), il sistema esegue le seguenti conversioni:
 - -1.0 si traduce in 2.5% (impulso di 0.5 ms per la rotazione minima).
 - 0.0 (centro) si traduce in 7.5% (impulso di 1.5 ms per la rotazione centrale).
 - +1.0 si traduce in 12.5% (impulso di 2.5 ms per la rotazione massima).
- **Aggiornamento dell'interfaccia:** in seguito all'invio dell'impulso, le coordinate vengono aggiornate a schermo sovrascrivendo i valori precedenti, restituendo un feedback visivo immediato all'utente.
- **Spegnimento sicuro:** in caso di terminazione volontaria (tasto 'q') o di interruzione anomala, un blocco `finally` richiama i metodi di arresto (`rotore_x.stop()` e `rotore_y.stop()`). L'interruzione fisica del segnale PWM impedisce ai motori di rimanere in tensione o di generare rumori anomali a programma concluso.

3.3.5 Modulo Gestione Telecamera + Rotori

Il presente modulo unisce le funzionalità dei due script precedenti, consentendo la gestione simultanea dell'acquisizione video e del puntamento hardware.

Funzionamento del Modulo

Lo script esegue due operazioni in parallelo:

- Visualizzazione in tempo reale del flusso video proveniente dalla telecamera **IMX219**, integrato con un HUD sovrapposto recante l'ora, la temperatura del dispositivo, la risoluzione e gli FPS.

- Controllo manuale dei due servomotori tramite l'input delle frecce direzionali da tastiera all'interno del terminale, permettendo la movimentazione della telecamera lungo gli assi orizzontale e verticale.

Inoltre, l'interfaccia HUD è stata estesa per visualizzare dinamicamente le coordinate attuali dei motori X e Y.



Figura 3.13: Rappresentazione del HUD finale, senza la presenza di bounding box, persone.

Librerie principali utilizzate

Oltre alle librerie già citate, il modulo si avvale in modo cruciale della libreria chiamata `threading`:

- **`threading` (Thread, Lock, Event):** risulta fondamentale poiché sia `GStreamer` (per la gestione video) sia `curses` (per l'intercettazione dell'input da tastiera) necessitano di operare all'interno di cicli infiniti. La libreria consente di superare le limitazioni di esecuzione lineare di Python, sdoppiando il processo e permettendo l'esecuzione parallela e concorrente delle due operazioni.
- **`gi.repository.Gst` e `gi.repository.GLib`:** impiegate per la costruzione della pipeline video in hardware e la gestione degli eventi multimediali.
- **`rpi_hardware_pwm` (HardwarePWM):** utilizzata per trasmettere il segnale fisico (Duty Cycle) ai due servomotori collegati ai Pin 12 e 13 del Raspberry Pi 5.
- **`curses`:** adottata per intercettare i comandi direzionali in tempo reale e per il rendering della console di comando nel terminale.

Architettura e Sincronizzazione

L'architettura software si fonda sull'impiego di due thread comunicanti:

Il Thread 1 (Video - thread_video): opera in background. Si occupa di eseguire la diagnostica hardware preliminare (es. `i2cdetect`, `libcamera-hello`), di tentare l'avvio delle pipeline candidate per aggirare eventuali anomalie del sistema operativo, di avviare lo streaming video (tramite la pipeline `libcamerasrc ! ... ! textoverlay ! autovideosink`) e di inizializzare il `GLib.MainLoop()`, utilizzando un timer di 2 secondi per il calcolo degli FPS e la lettura delle temperature.

Il Thread 2 (Rotori - _curses_main): viene eseguito nel Main Thread, in quanto la libreria `curses` necessita del controllo diretto della finestra del terminale. Gestisce l'inizializzazione dei canali PWM hardware, il disegno dell'interfaccia a riga di comando e l'ascolto ciclico dell'input da tastiera; alla ricezione di un comando, aggiorna le variabili logiche `pos_x` e `pos_y`, le converte in percentuale (Duty Cycle) e aziona fisicamente i motori.

La comunicazione sicura tra i processi concorrenti (Cross-Thread Communication) si avvale dei seguenti meccanismi di sincronizzazione per prevenire conflitti in memoria:

- **Lucchetto sulle posizioni (_pos_lock):** il Thread Rotori aggiorna in scrittura le coordinate X/Y alla pressione di un tasto, mentre il Thread Video esegue la lettura delle medesime variabili per aggiornare l'HUD. Il Lock garantisce l'accesso in mutua esclusione, impedendo letture e scritture simultanee.
- **Bandiera di chiusura (shutdown_event):** se l'utente innesca la terminazione (tasto 'q') nel terminale, il Thread Rotori attiva questo Event (`shutdown_event.set()`). Il Thread Video interroga lo stato di tale bandiera ciclicamente (ogni 500 ms) e, se attivata, chiude in modo sicuro l'istanza di `GStreamer`, terminando il processo. Analogamente, in caso di interruzione anomala di `GStreamer`, il Thread Video attiva la bandiera forzando la chiusura automatica del terminale.

3.3.6 Algoritmo Finale

Il presente algoritmo integra i moduli precedentemente descritti, realizzando l'obiettivo finale del progetto di tesi.

Funzionamento

Lo script esegue queste due operazioni principali:

- **Filtro dell'object detection:** il sistema isola e visualizza esclusivamente le bounding box associate alla classe persona.
- **Movimentazione automatica:** i rotori vengono azionati in modo dinamico per mantenere i soggetti rilevati all'interno del campo visivo della telecamera.



Figura 3.14: Frame video dove viene mostra il filtro attivo.

Librerie principali utilizzate

Oltre alle librerie precedentemente menzionate, l'implementazione si avvale di:

- **hailo:** impiegata per accedere alle rilevazioni generate dalla pipeline Hailo e applicare programmaticamente il filtro esclusivo sulla classe person.
- **Rimozione di curses:** tale libreria non risulta più necessaria e, pertanto, è stata eliminata in favore della gestione interamente automatizzata dei servomotori.

Logica Funzionamento

- **Filtro person_only**

```
roi = hailo.get_roi_from_buffer(buf) # estrae la Region Of Interest
detections = list(roi.get_objects_typed(hailo.HAILO_DETECTION)) # snapshot della lista

for det in detections:
    if det.get_label().lower() != "person":
        roi.remove_object(det) # rimuove dalla ROI → non verrà disegnata
    else:
        persons.append(det)
```

Figura 3.15: Porzione di codice che mostra come funziona il filtro.

Come evidenziato nel frammento di codice, il sistema estrae la regione di interesse (ROI) e la memorizza nella variabile **roi**, successivamente convertita in lista e assegnata a **detections**. Tramite un costrutto iterativo, vengono scartati tutti gli oggetti aventi un'etichetta (label) differente da "person". I restanti elementi validi vengono

conservati all'interno della lista **persons**, la quale costituirà l'insieme delle bounding box effettivamente renderizzate a schermo.

- **Controllo automatico dei servomotori:** Il tracciamento automatico si articola nelle seguenti fasi:
 - **Estrazione Bounding Box:** il modello YOLOv8n rileva i soggetti e restituisce le coordinate spaziali normalizzate nell'intervallo $[0, 1]$.

```
cx_sum = 0.0
cy_sum = 0.0
for det in persons:
    bbox = det.get_bbox()
    cx_sum += bbox.xmin() + bbox.width() / 2.0
    cy_sum += bbox.ymin() + bbox.height() / 2.0

cx = cx_sum / n    # media X
cy = cy_sum / n    # media Y
```

Figura 3.16: Porzione di codice che mostra il calcolo del centroide medio.

In presenza di molteplici rilevazioni, il sistema non focalizza il puntamento su un singolo individuo, ma calcola il baricentro geometrico dell'intero gruppo (posizionando, ad esempio, l'inquadratura al centro esatto tra due soggetti collocati agli estremi opposti).

- **Calcolo dell'errore di posizione:** viene quantificata la distanza spaziale del bersaglio rispetto al centro logico del fotogramma. Nello specifico, **error_x** = **cx** - 0.5 (dove un valore positivo indica una deviazione a destra) ed **error_y** = **cy** - 0.5 (dove un valore positivo indica una deviazione verso il basso).
- **Algoritmo di calcolo (PID):** la determinazione dell'angolo di spostamento dei rotori si fonda su un controllore operante su tre parametri:
 - * **P (Proporzionale):** reagisce all'errore posizionale istantaneo.
 - * **I (Integrale):** corregge l'errore persistente accumulato nel tempo (es. qualora il soggetto risulti costantemente decentrato).
 - * **D (Derivativo):** anticipa le variazioni, smorzando il movimento della telecamera quando il bersaglio si approssima al centro.

```

# Esempio: error_x = +0.20 (persona a destra), dt = 0.033s (30fps)

# P – Proporzionale
out_P = KP * error_x = 0.15 * 0.20 = +0.030

# I – Integrare (accumula l'errore nel tempo)
_pid_integral_x += error_x * dt = 0.20 * 0.033 = +0.0066
out_I = KI * _pid_integral_x = 0.02 * 0.0066 = +0.000132

# D – Derivativo (quanto sta cambiando l'errore)
deriv_x = (error_x - error_prev_x) / dt = (0.20 - 0.15) / 0.033 = +1.515
out_D = KD * deriv_x = 0.05 * 1.515 = +0.076

# Output totale
out_x = out_P + out_I + out_D = 0.030 + 0.000132 + 0.076 = +0.106

```

Figura 3.17: Esempio di come viene calcolato il movimento del rotore per l'asse x, lo stesso procedimento deve essere eseguito anche per l'asse y.

Per le finalità del progetto, le costanti operative sono state ricavate empiricamente e impostate come segue: $KP = 0.15$ (per una reattività rapida), $KI = 0.02$ (per compensare errori stazionari) e $KD = 0.05$ (per smorzare le oscillazioni).

- **Meccanismi di stabilizzazione:** al fine di garantire la fluidità visiva, sono stati implementati tre accorgimenti software:
 - * **Zona morta (Deadband):** se l'errore risulta inferiore al 3% dell'area del frame, i motori rimangono in stallo per prevenire micro-vibrazioni indotte dalle fluttuazioni fisiologiche della bounding box.
 - * **Anti-windup:** impedisce la crescita divergente della componente integrale qualora il bersaglio si trovi fisicamente fuori portata per periodi prolungati; un blocco limitatore (clamp) impostato a ± 0.5 frena l'accumulo matematico, prevenendo scatti anomali al rientro del soggetto.
 - * **Media Mobile Esponenziale (Smoothing EMA):** applica un filtro logico alle variazioni posizionali. Invece di raggiungere istantaneamente la coordinata target calcolata dal PID, il servomotore percorre il 12% della distanza rimanente a ogni frame, assicurando un movimento fluido e privo di asperità meccaniche.
- **Conversione PWM:** il valore posizionale risultante (appartenente all'intervallo $[-1.0, +1.0]$) viene infine convertito nel Duty Cycle del segnale PWM a 50 Hz, finalizzato all'azionamento elettromeccanico dei servomotori.

Capitolo 4

Casi e Risultati

Il presente capitolo analizza le prestazioni del dispositivo hardware, in termini di temperature registrate e fotogrammi per secondo (FPS) elaborati, durante l'esecuzione del software di tracciamento. Inoltre, verrà esaminato il comportamento del sistema all'interno di sei differenti scenari di test.

4.1 Metriche prestazionali: Analisi termica e FPS

I dati sono stati acquisiti durante l'esecuzione del software di tracciamento automatico. Le misurazioni sono state campionate con un intervallo di tre secondi all'interno di una finestra temporale compresa tra le 14:08:58 e le 18:04:43, per una durata complessiva di 3 ore, 55 minuti e 45 secondi.

Temperatura

Di seguito è riportato un grafico che illustra la variazione della temperatura.

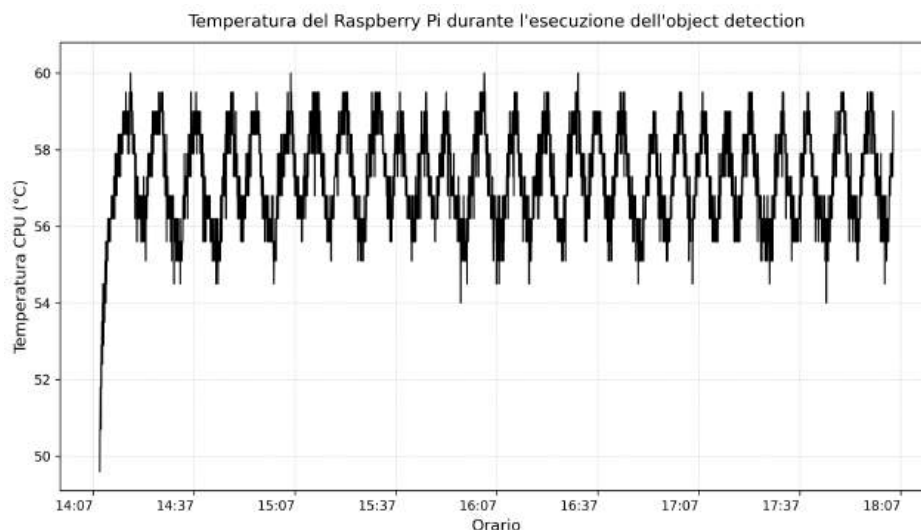


Figura 4.1: Andamento della temperatura del Raspberry Pi.

Osservazioni: Nei primi minuti di esecuzione si registra un rapido innalzamento termico. La temperatura passa da uno stato di riposo (circa 49,6 °C) fino a raggiungere la soglia dei 60 °C, una diretta conseguenza dell'avvio del carico computazionale generato dalla pipeline video e dal modello di object detection. Dopo il picco iniziale, la temperatura entra in una fase ciclica e regolare, mantenendosi per lo più all'interno di un intervallo compreso tra i 54,5 °C e i 59,5 °C. Dal pattern del grafico si deduce che la ventola del sistema di dissipazione aumenta il proprio regime di rotazione in prossimità della soglia dei 60 °C, abbattendo rapidamente la temperatura fino al limite inferiore, per poi ridurre la velocità e innescare così un nuovo ciclo di riscaldamento. In questo modo viene garantita la stabilità operativa e l'assenza di throttling; tuttavia, il dato più rilevante è la stabilità del sistema sul lungo periodo. La temperatura massima registrata raggiunge raramente i 60 °C, un valore nettamente inferiore alla soglia critica di thermal throttling del Raspberry Pi 5 (generalmente impostata a 80-85 °C). Ciò dimostra che il sistema di raffreddamento previene efficacemente il surriscaldamento, garantendo l'elaborazione del flusso video e l'inferenza della rete neurale senza alcuna degradazione prestazionale causata da limitazioni termiche.

Di seguito è riportata una tabella che riassume le principali statistiche estratte dal grafico e dai log relativi alla temperatura.

Parametro	Valore
Numero di rilevazioni	4716
Durata del monitoraggio	3 h 55 min 45 s
Temperatura iniziale	49.6 °C
Temperatura finale	59.0 °C
Temperatura minima	49.6 °C
Temperatura massima	60.0 °C
Temperatura media	57.28 °C
Mediana	57.3 °C
Deviazione standard	1.22 °C

Tabella 4.1: Statistiche descrittive delle temperature rilevate durante l'esecuzione del programma di object detection.

Fotogrammi per Secondo (FPS)

Di seguito è riportato un grafico che illustra la variazione degli FPS.

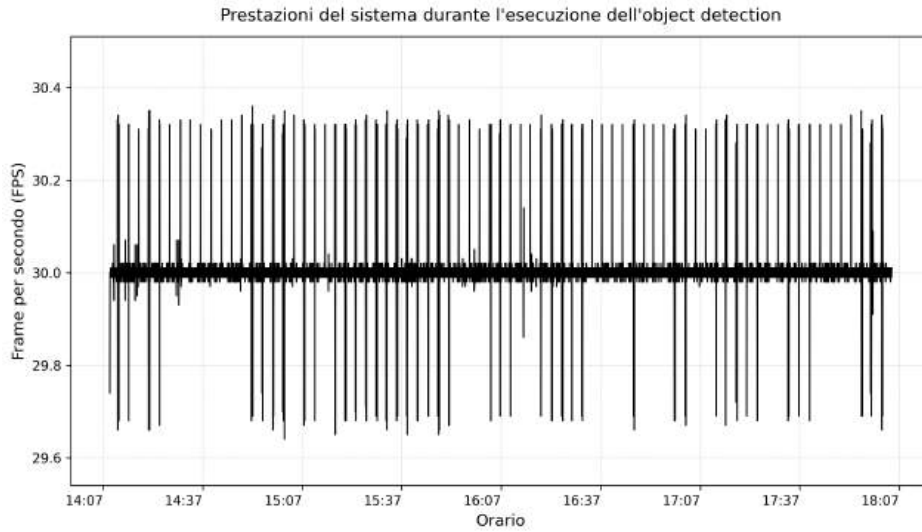


Figura 4.2: Andamento del numero di frame elaborati al secondo (FPS) dal sistema durante l'esecuzione del programma di object detection.

Osservazioni: Dal grafico si osserva che il sistema mantiene stabilmente il framerate nominale di circa 30 FPS del flusso utilizzato, senza evidenti degradazioni nel corso del test. È pertanto possibile dedurre che l'elaborazione demandata alla NPU Hailo-8 garantisca prestazioni real-time continue. Il carico computazionale generato dal modello YOLOv8n viene gestito interamente dall'hardware dedicato, evitando ritardi di elaborazione anche sotto sforzo prolungato. Le oscillazioni periodiche visibili nel grafico risultano circoscritte in un intervallo estremamente ridotto e trascurabile. Questi scostamenti minimi non indicano un degrado prestazionale, ma sono imputabili allo scheduling dei thread del sistema operativo e alle tolleranze di calcolo del timer asincrono incaricato del monitoraggio degli FPS.

Di seguito è riportata una tabella che riassume le principali statistiche estratte dal grafico e dai log relativi agli FPS.

Parametro	Valore
Numero di rilevazioni	4715
Durata del monitoraggio	3 h 55 min 42 s
FPS minimo	29.64
FPS massimo	30.36
FPS medio	30.01
Mediana	30.00
Deviazione standard	0.07

Tabella 4.2: Statistiche descrittive degli FPS rilevati durante l'esecuzione del programma di object detection.

4.2 Casi Sperimentati

In questa sezione, per ciascun caso di test esaminato, verrà fornita una breve descrizione introduttiva seguita dall'analisi delle relative osservazioni emerse durante la fase di esecuzione.

Tracciamento Singola Persona In questo caso viene valutata la capacità del sistema di effettuare il tracking di una singola persona all'interno del campo visivo. Questo rappresenta il caso più semplice e costituisce lo scenario di base.



Figura 4.3: Frame in cui è presente una singola persona tracciata dal sistema.

Osservazioni: Il tracciamento risulta costante e l'intera figura della persona viene mantenuta prevalentemente al centro del campo visivo della telecamera. Il movimento dell'inquadratura si presenta leggermente a scatti. Quando la persona rimane ferma al centro del campo visivo non si registrano effetti di tremolio, né sull'asse orizzontale né su quello verticale.

Tracciamento di più persone Questo scenario valuta la capacità del sistema di seguire più persone simultaneamente, analizzandone il comportamento in una situazione iniziale in cui un singolo gruppo si divide in due sotto-gruppi diretti in direzioni opposte. Tale dinamica rappresenta la situazione più frequente all'interno di una competizione sportiva a squadre.



Figura 4.4: Due frame che illustrano la divisione di un gruppo iniziale di persone.

Osservazioni: Come si evince dal primo frame, quando il gruppo è unito il centro logico individuato dall'algoritmo coincide con il centro del campo visivo. Nel momento in cui il gruppo si separa, il sistema tenta di mantenere tutti i soggetti all'interno dell'inquadratura, assegnando tuttavia una priorità maggiore al gruppo più numeroso. Di conseguenza, il gruppo con meno elementi esce dal campo visivo e la telecamera si riposiziona inquadrando centralmente la formazione più consistente.

Tracciamento di persone tra primo piano e sfondo Questo test esamina la capacità del sistema di gestire soggetti dislocati su piani di profondità differenti. L'obiettivo è determinare in che misura il controllo dell'inquadratura venga influenzato dalla vicinanza di un individuo all'obiettivo della telecamera. Il test è stato condotto impiegando esclusivamente due persone.



Figura 4.5: Frame in cui sono presenti una persona in primo piano e una sullo sfondo, entrambe tracciate dal sistema.

Osservazioni: Inizialmente entrambe le figure vengono tracciate regolarmente. Tuttavia, si osserva che, qualora la persona in secondo piano rimanga ferma mentre quella in primo piano è in movimento, il sistema assegna la priorità al soggetto sullo sfondo, causando l'uscita dell'altro individuo dall'inquadratura. Al contrario, se entrambe le figure si muovono in direzioni opposte, il sistema privilegia il tracciamento della persona in primo piano.

Persona che entra nell'inquadratura Questo scenario analizza la reazione del sistema all'ingresso di un nuovo soggetto all'interno del campo visivo. Si parte da una situazione iniziale di inquadratura vuota, seguita dall'ingresso di un individuo. Il caso di test è funzionale a valutare la latenza e la fluidità di aggancio del bersaglio.



Figura 4.6: Frame illustrativo dell'ingresso di una persona nel campo visivo della telecamera.

Osservazioni: All'ingresso della persona nel campo visivo, si osserva che il sistema orienta l'inquadratura verso di essa in maniera graduale, accompagnandone il movimento fino a stabilizzarla al centro della scena.

Persona in uscita dall'inquadratura In questo caso si valuta il comportamento del sistema quando una persona esce dall'inquadratura mentre un'altra vi permane. Il test è utile per comprendere le logiche di compensazione a fronte di transiti momentanei di giocatori secondari.



Figura 4.7: Frame in cui sono visibili due persone: una statica all'interno del campo visivo e una in fase di uscita.

Osservazioni: Il sistema asseconda parzialmente il soggetto in uscita, cercando di mantenere entrambe le figure nel campo visivo. Non appena l'individuo esce definitivamente dall'inquadratura, il sistema si ricalibra riposizionando centralmente la persona rimasta.

Scatto di una persona Questo test esamina la reattività del sistema nel tracciare i movimenti ad alta intensità (scatti) di un singolo individuo. Tale verifica è cruciale per determinare se l'attuale prototipo possieda le specifiche dinamiche necessarie a seguire le azioni fulminee tipiche di una partita competitiva.



Figura 4.8: Frame che mostra il tracciamento di una persona durante uno scatto.

Osservazioni: Durante la fase di validazione sono state sperimentate due intensità di scatto: una moderata e una ad alta velocità. Nel caso dello scatto moderato, il sistema riesce a inseguire il movimento, mantenendo il soggetto a fuoco pur confinandolo ai margini dell'inquadratura. Nel caso dello scatto ad alta velocità, invece, sebbene il sistema riesca inizialmente ad agganciare il movimento, subisce poco dopo una perdita del tracciamento a causa dell'eccessiva rapidità dell'individuo, che fuoriesce definitivamente dal campo visivo.

Capitolo 5

Conclusioni

5.1 Limiti della Ricerca

L'analisi dei risultati ottenuti durante la fase di test ha evidenziato diverse limitazioni tecniche e operative del sistema sviluppato.

Per quanto concerne i limiti meccanici e il tracciamento dinamico, i test relativi agli scatti rapidi hanno dimostrato che i micro servomotori SG90, pur risultando economici e funzionali per la gestione di movimenti standard, presentano limitazioni intrinseche in termini di velocità angolare. Tali vincoli impediscono al sistema di assecondare variazioni di traiettoria repentine o scatti ad alta velocità, causando la fuoriuscita del soggetto dall'inquadratura e la conseguente perdita del tracciamento.

Un'ulteriore criticità riguarda la gestione delle occlusioni e le priorità logiche: nei casi in cui i soggetti si sovrappongano, come avviene nel rapporto tra primo piano e sfondo, o qualora i gruppi si dividano, l'algoritmo basato sul calcolo del centroide medio assegna automaticamente la priorità alla massa di persone più numerosa. Questa logica, sebbene efficace a livello puramente geometrico, risulta inefficace rispetto al reale contesto tattico della partita, con il rischio di trascurare l'azione saliente, come nel caso di un portatore di palla che si distacca dal gruppo.

Infine, l'attuale implementazione si avvale di una classificazione generica, applicando un filtro esclusivamente alla classe "person" del modello YOLOv8n. Di conseguenza, il sistema non è in grado di discriminare tra i giocatori della squadra di casa, gli avversari, gli arbitri o gli eventuali spettatori a bordo campo, limitando fortemente la specificità e la profondità dell'analisi tattica.

5.2 Sviluppi Futuri

Al fine di superare le criticità riscontrate e ampliare le potenzialità del sistema, si delineano diversi ambiti di sviluppo futuro.

La sostituzione dei micro servomotori SG90 con motori passo-passo (*stepper motors*) o servomotori *brushless* ad alte prestazioni garantirebbe un controllo angolare più rapido, fluido e preciso, eliminando le perdite di tracciamento durante le fasi di gioco più concitate.

L'affinamento dell'infrastruttura software prevede il *fine-tuning* del modello YOLO su dataset specifici per la pallacanestro. Ciò consentirebbe il riconoscimento e il tracciamento della palla, la classificazione dei giocatori in base al colore della divisa e l'esclusione automatica degli arbitri e degli spettatori. Di conseguenza, risulterebbe possibile sviluppare un algoritmo che utilizzi la posizione della palla, congiuntamente a una strategia di associazione spazio-temporale, per identificare e assegnare la priorità al giocatore attivamente coinvolto nell'azione.

L'architettura IoT progettata potrebbe essere scalata impiegando molteplici nodi (Raspberry Pi integrati con telecamere) disposti lungo il perimetro del campo da gioco. La fusione dei dati provenienti da più angolazioni permetterebbe la ricostruzione delle traiettorie in uno spazio tridimensionale, garantendo una copertura totale e offrendo metriche tattiche avanzate, simulando i sistemi professionali ad alto costo. È opportuno precisare, tuttavia, che per ottenere una ricostruzione 3D accurata non è sufficiente la mera installazione di più dispositivi: a livello software risulterà necessaria l'implementazione di procedure avanzate per la calibrazione delle camere, la sincronizzazione temporale dei flussi, l'associazione logica delle singole rilevazioni visive provenienti dai vari nodi e i successivi calcoli di triangolazione spaziale.

5.3 Conclusioni

Il presente progetto di tesi ha dimostrato la fattibilità di un sistema IoT di *edge computing* per l'analisi video sportiva, avvalendosi esclusivamente di componenti hardware a basso costo e algoritmi di intelligenza artificiale leggeri.

L'integrazione del Raspberry Pi 5 con la NPU Hailo-8 (AI HAT+) ha consentito di eseguire il modello YOLOv8n localmente, elaborando il flusso video in tempo reale senza la necessità di ricorrere a server *cloud* esterni. L'analisi prestazionale ha confermato l'elevata efficienza dell'architettura: il sistema ha mantenuto un'elevata stabilità nell'elaborazione video (30 FPS costanti) durante sessioni di test prolungate (circa quattro ore), operando all'interno di un regime termico controllato (con un picco massimo di 60.0°C) che ha scongiurato qualsiasi fenomeno di *thermal throttling*. Inoltre, l'algoritmo sviluppato per il controllo integrato, il quale unisce l'estrazione delle *bounding box* a un controllore logico per la movimentazione automatica dei rotori, ha evidenziato un funzionamento reattivo, inseguendo autonomamente i soggetti all'interno del campo visivo.

In conclusione, sebbene sussistano margini di miglioramento legati ai limiti meccanici dei servomotori e alla necessità di impiegare reti neurali addestrate ad hoc, l'infrastruttura progettata si pone come un'alternativa accessibile, scalabile e autonoma ai proibitivi sistemi commerciali di tracciamento sportivo, aprendo la strada a future applicazioni avanzate per le società sportive dilettantistiche e semi-professionistiche.

Bibliografia

- [1] Weisong Shi et al. “Edge computing: Vision and challenges”. In: *IEEE internet of things journal* 3.5 (2016), pp. 637–646.
- [2] Karen Rose, Scott Eldridge, Lyman Chapin et al. “The internet of things: An overview”. In: *The internet society (ISOC)* 80.15 (2015), pp. 1–53.
- [3] Yali Amit, Pedro Felzenszwalb e Ross Girshick. “Object detection”. In: *Computer vision: A reference guide*. Springer, 2021, pp. 875–883.
- [4] Yihui He et al. “Bounding Box Regression With Uncertainty for Accurate Object Detection”. In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. Giu. 2019.
- [5] Christoph Bartneck et al. *An introduction to ethics in robotics and AI*. Springer, 2021.
- [6] Chip Huyen. *Designing machine learning systems*. O’Reilly Media, Inc., 2022.
- [7] Maad M Mijwel, Adam Esen e Aysar Shamil. “Overview of neural networks”. In: *Babylonian Journal of Machine Learning* 2023 (2023), pp. 42–45.
- [8] Puja Bharati e Ankita Pramanik. “Deep learning techniques—R-CNN to mask R-CNN: a survey”. In: *Computational Intelligence in Pattern Recognition: Proceedings of CIPR 2019* (2019), pp. 657–668.
- [9] Tausif Diwan, G Anirudh e Jitendra V Tembhurne. “Object detection using YOLO: challenges, architectural successors, datasets and applications”. In: *Multimedia Tools and Applications* 82.6 (2023), pp. 9243–9275.
- [10] Sara Battelini. “Computer vision for detecting and tracking players in basketball videos”. Tesi di laurea mag. Politecnico di Torino, 2020.
- [11] Ivan Ćirić et al. “Computer Vision-Driven Framework for IoT-Enabled Basketball Score Tracking”. In: *Computers* 14.11 (2025), p. 469.
- [12] Changjia Tian et al. “Use of machine learning to automate the identification of basketball strategies using whole team player tracking data”. In: *Applied Sciences* 10.1 (2019), p. 24.
- [13] Juan Terven, Diana-Margarita Córdova-Esparza e Julio-Alejandro Romero-González. “A comprehensive review of yolo architectures in computer vision: From yolov1 to yolov8 and yolo-nas”. In: *Machine learning and knowledge extraction* 5.4 (2023), pp. 1680–1716.

- [14] Wim Taymans et al. “Gstreamer application development manual (1.7. 0.1)”. In: *Dostupné z: <http://gstreamer.freedesktop.org/data/doc/gstreamer/head/manual/manual.pdf> (leden až březen 2013)* (2013).
- [15] Ben Nuttall. *Gpiozero Documentation*. 2017.

Ringraziamenti

Desidero ringraziare il laboratorio CGLab per avermi fornito il materiale e il supporto necessari alla realizzazione di questa tesi.

Un ringraziamento va alla mia famiglia per essermi stata sempre vicina, specialmente in questi ultimi periodi di stress. In particolare, ringrazio mia sorella Laura per essersi prestata come "modella" per i casi sperimentali, insieme alla nostra amica Manila.

Ringrazio i miei colleghi universitari del gruppo H-PET: nonostante i percorsi si siano divisi negli anni successivi, abbiamo continuato a frequentarci e a supportarci a vicenda durante la preparazione dei vari esami.

Infine, ringrazio i miei amici per essere stati sempre presenti, per avermi supportato e sopportato al di fuori dell'ambito universitario; ogni nostra uscita ha rappresentato per me una preziosa e indispensabile boccata d'aria fresca.